

Lesson 1 – Big Picture: What Is This “Trustworthy Voice” Project?

1. Lesson Summary

In this lesson, the student will get a **high-level overview** of the research paper “*Human voices communicating trustworthy intent: A demographically diverse speech audio dataset*” and the project they will eventually reproduce.

We’ll explain in plain language:

- What the paper is trying to discover: *Can we hear trust in someone’s voice, and can a computer detect it?*
- What the dataset looks like: 96 speakers of different ages and ethnicities, each reading sentences in a **neutral voice** and with a **trustworthy intent**, giving 1,152 audio files.
- How the researchers turned raw recordings into numbers (acoustic features like pitch and loudness).
- How they trained machine learning models (Random Forest & Logistic Regression) to classify “neutral” vs “trustworthy” speech, reaching about **70% accuracy** and AUC around **0.71–0.78**.

By the end of this lesson, the student should understand **what the final goal is** and how all the pieces fit together: recordings → features → labels → ML model → results.

2. Key Points

- The paper provides a **public speech dataset**: 1,152 utterances (short sentences) recorded by 96 speakers.
 - Each speaker says sentences **twice**: once in their **neutral** voice and once trying to sound **trustworthy**.
 - Speakers come from **different ethnicities** (white, Black, South Asian) and **age groups** (younger 18–45, older 60+), and both sexes.
 - All recordings are **standardized** (same format and sampling rate) and cut into separate files for each sentence.
 - For each audio file, the researchers extracted **acoustic features** such as:
 - Fundamental frequency (F0, perceived as pitch)
 - Amplitude (loudness)
 - Harmonics-to-noise ratio (HNR)
 - Jitter, shimmer
 - Cepstral peak prominence (CPP)
 - Long-term average spectrum (LTAS)
 - They used these features as **inputs** to two ML models: **Random Forest** and **Logistic Regression**.
 - They evaluated models using **accuracy, precision, recall, F1**, and **AUC**, and got around **70% accuracy** overall.
 - The student’s long-term goal:
 - Download the dataset & code
 - Understand the steps
 - Re-run and slightly extend the experiments in Python.
-

3. Real-World Examples or Stories

1. **Phone call from a stranger** Imagine you get a call from someone saying, “Hi, I’m from your bank, I just need your card number.” Even before checking the facts, your *ears* judge if they sound trustworthy: do they sound calm, confident, and helpful — or rushed and nervous?
 2. **Teacher vs scammer voice** Compare your favorite teacher saying, “Don’t worry, I’ll help you,” with a random online scammer. Their **words** might be similar, but the **voice** feels different. That emotional “feel” is partly what this dataset captures.
 3. **Virtual assistants** Voice assistants (Siri, Alexa, Google Assistant) are designed to sound friendly and trustworthy. Their voice pitch, speed, and clarity are tuned so people feel comfortable.
 4. **Customer service** When you call tech support, the agent’s voice can make you feel safe (“They know what they’re doing”) or annoyed. Companies care a lot about this because it impacts customer satisfaction.
 5. **Robots & social AI** As we design social robots or AI that talk to people (elder care robots, therapy chatbots with voice), we need to understand **what makes a voice sound trustworthy** so we don’t accidentally create creepy or misleading behavior.
-

4. Terminology Explained

- **Utterance** – One spoken sentence or phrase, saved as a single audio file.
 - **Neutral intent** – The speaker just talks normally, without trying to sound extra anything (friendly, scary, etc.).
 - **Trustworthy intent** – The speaker *tries* to sound like someone you can trust (warm, reliable, honest).
 - **Acoustic feature** – A number describing some property of the sound (e.g., pitch, loudness, voice quality).
 - **Fundamental frequency (F0)** – How fast the vocal folds vibrate; we hear it as **pitch** (high vs low voice).
 - **Harmonics-to-noise ratio (HNR)** – How “clean” or “noisy/rough” a voice is. Higher HNR = cleaner, less noisy voice.
 - **Jitter** – Tiny random changes in pitch, reflecting instability in the vocal folds.
 - **Shimmer** – Tiny random changes in loudness, reflecting instability in vocal intensity.
 - **Dataset** – A structured collection of data (here: audio files + speaker info + acoustic features + labels).
 - **Label** – The thing we want the model to predict (here: neutral vs trustworthy).
 - **Classifier** – A machine learning model that tries to assign each example to a category (class).
 - **Random Forest** – A model that uses many decision trees and averages their predictions.
 - **Logistic Regression** – A simple, widely-used linear model for binary classification.
 - **AUC (Area Under the ROC Curve)** – A number between 0 and 1 that shows how well the model separates the two classes (0.5 = random, 1.0 = perfect).
-

5. How It Works (Real-World Pipeline)

Here’s the **full pipeline** of the paper, simplified:

1. **Collect speech recordings**
 - 96 people speak 20 sentences each.
 - Each sentence is recorded twice: neutral + trustworthy.
2. **Standardize the audio**
 - Convert all recordings to the same format: WAV, 48 kHz sampling rate, same loudness level (67 dB).
3. **Segment into utterances**
 - Each long recording session is cut into **single-sentence files** so every sentence is a separate audio file.

4. Extract acoustic features

- Tools like **VoiceLab** compute pitch, HNR, jitter, shimmer, CPP, LTAS, etc., for each file.
- Result: each audio file becomes a **row in a table**, with columns like **mean_F0**, **HNR**, **duration**, etc.

5. Set labels

- Each row is labeled as **neutral** or **trustworthy** intent (0 or 1).

6. Train machine learning models

- Use acoustic features as **inputs (X)** and intent labels as **outputs (y)**.
- Train **Random Forest** and **Logistic Regression** using a strategy called **leave-one-speaker-out cross-validation** (train on everyone except one speaker, test on the left-out speaker; repeat).

7. Evaluate performance

- Check how often the models can correctly guess whether a file is neutral vs trustworthy.
- Accuracy $\approx 70\%$, AUC ≈ 0.71 – 0.78 overall and across subgroups.

8. Interpret features

- Use **feature importance** (Random Forest) and coefficients (Logistic Regression) to see which acoustics matter most.
- Pitch, pitch variability (SD F0), HNR, shimmer, and CPP come out as important for distinguishing trustworthy intent.

In the **full 10-lesson course**, we'll go deep into each step. Lesson 1 just builds the mental map.

6. Practice Exercises

Exercise 1 – Why Does Voice Matter? In 3–5 sentences, explain **why** you think a person's voice (not their words) can affect whether you trust them.

Exercise 2 – Label vs Feature You have this small table:

Example	Mean Pitch (Hz)	Duration (s)	Intent
A	180	1.6	Trustworthy
B	150	1.8	Neutral

a) Which column(s) are **features**? b) Which column is the **label**?

Exercise 3 – Two Intents Imagine you are reading the sentence: "I will help you with your homework."

Describe how you might say it in: a) Neutral intent. b) Trustworthy intent.

Mention at least **two** acoustic changes (e.g., pitch, speed, loudness).

Exercise 4 – Pipeline Ordering Put these steps in the correct order for this project:

1. Extract acoustic features
2. Train machine learning models
3. Record voices
4. Evaluate model performance

5. Label each utterance as neutral or trustworthy

Exercise 5 – Reasoning About Features Suppose a model learns that higher pitch and higher HNR are associated with trustworthy speech in this dataset.

Explain in 2–3 sentences why this might make sense or not make sense in real life.

7. Solutions / Model Answers

Exercise 1 – Sample Answer A person's voice carries emotion and attitude. If someone speaks calmly, clearly, and confidently, we tend to feel they know what they're doing and care about us. If they sound rushed, shaky, or bored, we might feel uneasy, even if their words are nice. So, our brain uses voice as a shortcut for deciding if someone is safe and trustworthy.

Exercise 2 – Answer

a) **Features:**

- Mean Pitch (Hz)
- Duration (s)

b) **Label:**

- Intent

The model uses the features (numbers about the sound) to predict the label (neutral or trustworthy).

Exercise 3 – Answer

a) **Neutral intent** – Speak in your normal everyday voice, medium speed, medium volume, not putting extra emotion into it.

b) **Trustworthy intent** – Maybe slightly slower, clearer articulation, steady volume, and a warmer tone. You might raise your pitch a little at the end to sound friendly and supportive, and avoid sounding flat or bored.

Exercise 4 – Answer (Correct Order)

1. Record voices
 2. Label each utterance as neutral or trustworthy
 3. Extract acoustic features
 4. Train machine learning models
 5. Evaluate model performance
-

Exercise 5 – Sample Answer

Higher pitch and higher HNR might make a voice sound more energetic, bright, and clear, which people could interpret as friendliness and honesty. However, this is not universal. In some cultures or situations, a deeper voice might be seen as more trustworthy. The model is only capturing patterns specific to this dataset, not universal laws of human trust.

8. Q&A – Common Student Questions

1. **Q:** Why do we need both neutral and trustworthy versions of each sentence? **A:** Using the *same words* but different intents controls for the content. Any differences the model learns are more likely due to **how** something is said (acoustics), not **what** is said.
2. **Q:** Why not just ask people if a voice is trustworthy instead of modeling it? **A:** Human ratings are important, but computers can process large datasets and reveal patterns we might miss. Also, a model can be used later in applications like voice assistants or social robots.
3. **Q:** Isn't it creepy to teach a model to sound trustworthy? **A:** It *can* be misused, but the goal here is scientific understanding and **ethical** applications (e.g., making customer service less stressful or helping people with communication difficulties). Ethical use is a big part of AI.
4. **Q:** What's the difference between pitch and loudness? **A:** Pitch is how **high or low** a voice sounds (like musical notes). Loudness is how **strong or quiet** the sound is.
5. **Q:** What does "trustworthy intent" mean exactly? **A:** It means the speaker is *trying* to sound like someone you can trust — based on their own idea of what a trustworthy voice sounds like.
6. **Q:** Why do we use features like HNR, jitter, and shimmer? **A:** These capture aspects of **voice quality** (smooth vs rough, stable vs shaky). Past research shows these are linked to how people perceive voices (e.g., healthy vs unhealthy, confident vs nervous).
7. **Q:** Why are there different ethnicities and age groups in the dataset? **A:** To avoid "white western individualist bias" and to make the results more generalizable across different kinds of speakers.
8. **Q:** What is leave-one-speaker-out cross-validation? **A:** It's a way to test generalization: train on all speakers except one, test on that one, then repeat for each speaker. This checks if the model works on *new people*, not just the ones it has seen.
9. **Q:** Why not use deep learning instead of Random Forest and Logistic Regression? **A:** Traditional models are simpler, easier to explain, and work well with small-to-medium datasets of hand-crafted features. Deep learning might help later, but this study focuses on clear, interpretable baselines.
10. **Q:** Is 70% accuracy "good"? **A:** It's better than random guessing (50%), so the model is clearly picking up real patterns. But it's not perfect, which makes sense: trust is complex and not fully determined by voice acoustics.

9. Quiz (10 Questions + Answers)

Q1. What is the main goal of the paper? a) To recognize specific words in speech b) To detect trustworthy intent in voices c) To translate languages d) To separate background noise

Answer: b) – The study focuses on detecting trustworthy intent in speech.

Q2. How many total audio recordings are in the dataset? a) 96 b) 576 c) 1,152 d) 20

Answer: c) – There are 1,152 recordings from 96 speakers.

Q3. What are the two main speech intents in this dataset? **Answer:** Neutral intent and Trustworthy intent.

Q4. Which of the following is **not** an acoustic feature mentioned in the paper? a) Jitter b) Shimmer c) Cepstral peak prominence (CPP) d) Translation probability

Answer: d) – Translation probability is unrelated; the others are acoustic features.

Q5. Why did the researchers include speakers from different ethnicities and age groups? **Answer:** To make the dataset more diverse and reduce bias, so the results generalize beyond a single demographic group.

Q6. What is the label that the models try to predict? a) The sentence ID b) Speaker ID c) Age d) Intent (neutral vs trustworthy)

Answer: d) – The label is the speech intent.

Q7. Which models were used to classify trustworthy intent? **Answer:** Random Forest and Logistic Regression.

Q8. What does an AUC of 0.75 roughly mean? **Answer:** It means that, on average, the model has a 75% chance of ranking a random trustworthy example higher than a random neutral example — better than random (0.5), but not perfect (1.0).

Q9. Why do we standardize the audio files (same format, sampling rate, and loudness)? **Answer:** To remove technical differences between recordings so that the model focuses on **meaningful** differences in voice, not on file format or volume.

Q10. True or False: This dataset includes actors pretending to be different characters. **Answer:** False – The speakers are **untrained**, meaning they are not professional actors.

10. Mini Practice Project (With Downloadable .zip)

Title: *Mini Voice Trust Classifier (Synthetic Data)*

Goal: Let the student **practice the core idea** of the paper on a tiny, easy-to-run dataset:

- Each row is a synthetic “utterance” with:
 - `duration_seconds`
 - `mean_pitch_hz`
 - `hnr_db`
 - `intent_label` (0 = neutral, 1 = trustworthy)
- They will train **Logistic Regression** to predict `intent_label` and see the accuracy & model coefficients.

I’ve created a `.zip` file for you that contains:

- `lesson1_mini_voice_trust_project/mini_voice_trust_dataset.csv`
- `lesson1_mini_voice_trust_project/train_logistic_regression.py`
 - Every single line has a comment explaining what it does.
- `lesson1_mini_voice_trust_project/README.txt`
 - Step-by-step instructions to run the project.

What the student should do:

1. Unzip the file on their computer.
2. Open a terminal in the `lesson1_mini_voice_trust_project` folder.
3. Install dependencies (once):

```
pip install pandas scikit-learn
```

4. Run:

```
python train_logistic_regression.py
```

5. Read the printed **accuracy** and **coefficients**.
6. Try simple modifications, for example:
 - Change the test size
 - Remove one feature and see how accuracy changes

This mirrors the paper's idea in a **toy environment**: acoustic features → logistic regression → intent prediction.

11. References (Starter Reading / Watching)

- Original dataset and code (OSF repository – as cited in the paper):
 - “Trustworthy Intent in Speech (TIS) Corpora Dataset” on OSF
- The paper itself (for later lessons):
 - *Human voices communicating trustworthy intent: A demographically diverse speech audio dataset* (Scientific Data, 2025).
- Introductory materials (for the student):
 - scikit-learn’s “Getting Started” guide (Logistic Regression section)
 - Short YouTube videos on:
 - “What is pitch in sound?”
 - “Basic introduction to machine learning classification”
 - Any high-school-friendly overview of:
 - Random Forests
 - Logistic Regression

(We’ll bring in more targeted references — Praat, VoiceLab, detailed audio ML resources — in later lessons.)

12. Additional Information / Teacher Tips

- **Concept first, math later** At this stage, don’t worry about logistic regression formulas. Focus on: *we have inputs* → *the model learns patterns* → *we can predict labels and measure how good we are*.

- **Use their own voice** A fun warm-up is to have the student record themselves saying one sentence neutrally and then with trustworthy intent. Let them *feel* how they change pitch, speed, and loudness.
- **Connect to ethics early** Raise the question: “Is it okay to build an AI that knows how to sound trustworthy?” Let them think about misleading voices, scams, and also helpful uses (therapy bots, accessibility).

Lesson 2 – How Voices Become Numbers: Acoustic Features 101

1. Lesson Summary

In this lesson, the student will learn **how a human voice recording turns into numbers** that a machine learning model can understand.

We’ll focus on the kinds of **acoustic features** used in the paper, such as:

- **Pitch (F0) and pitch variability**
- **Duration**
- **Harmonics-to-Noise Ratio (HNR)**
- **Jitter & shimmer**
- **Cepstral Peak Prominence (CPP)**
- **Long-Term Average Spectrum (LTAS)**

The goal is not to memorize every formula, but to build a **clear intuition**:

“Each audio file → a row in a table → columns are features describing how that voice sounds.”

By the end of this lesson, the student should be able to **read a feature table** and explain, in their own words, what the key features mean and how they might relate to sounding trustworthy.

2. Key Points

- Audio is a **continuous waveform**, but computers store it as many **samples per second** (e.g., 48,000 samples per second).
- From a waveform, we can measure **pitch**, **loudness**, **duration**, and many kinds of **voice quality**.
- The paper’s dataset computes a set of **acoustic features** for each utterance (one sentence spoken).
- **Pitch (F0)** = how high or low the voice sounds; **SD F0** = how much pitch wiggles over time.
- **Jitter** and **shimmer** measure tiny random changes in pitch and loudness, related to how stable or shaky a voice is.
- **HNR** and **CPP** measure how “clear” or “harmonic” the voice is vs. noisy/rough.
- **LTAS** summarizes the average distribution of energy across frequencies (low vs high tones).
- Each utterance becomes a **feature vector**: `[mean_F0, SD_F0, HNR, shimmer, CPP, duration, ...]`.
- These vectors are used as inputs to machine learning models to predict **intent** (neutral vs trustworthy).

3. Real-World Examples or Stories

1. **Singing a note** When someone sings a steady note “aaaa,” the **pitch** (F0) stays nearly constant. If they wobble, vibrato appears as **variation in F0** and maybe in loudness (shimmer).
2. **Nervous vs calm presentation** When you’re nervous giving a speech, your voice might shake slightly: irregular pitch (jitter) and loudness (shimmer) increase. Calm speakers often have smoother, more stable voices.

3. **Phone call in a noisy street** If you call someone from a crowded street, your voice may be less clear and more noisy. That's related to **HNR** being lower (more noise mixed into the signal).
 4. **"Warm" vs "thin" voice** Some voices feel warm and rich (more low- and mid-frequency energy), while others feel thin (more high-frequency energy). This is partly what **LTAS** and **CPP** are capturing.
 5. **Time vs frequency views** Looking at a waveform is like looking at **how the air moves over time**; looking at a **spectrum** is like asking: "Which musical notes (frequencies) are present and how strong are they?" Both are useful to describe a voice.
-

4. Terminology Explained

- **Waveform** – The raw audio signal, showing how air pressure changes with time.
 - **Sample rate** – How many audio samples are taken per second (e.g., 48,000 Hz).
 - **Utterance** – One spoken sentence or phrase, saved as a separate audio file.
 - **Feature** – A single measurable property (number) describing the sound (e.g., mean pitch).
 - **Feature vector** – A list of features for one example (row), e.g., `[F0, HNR, duration, ...]`.
 - **Frame** – A short slice of audio (e.g., 20–40 ms). Many features are first computed per frame and then averaged.
 - **Fundamental frequency (F0)** – The main vibration rate of the vocal folds; we hear it as pitch.
 - **SD F0** – Standard deviation of F0; how much pitch changes over time.
 - **Jitter** – Small, random fluctuations in pitch between successive cycles.
 - **Shimmer** – Small, random fluctuations in loudness between successive cycles.
 - **HNR (Harmonics-to-Noise Ratio)** – Ratio of periodic (harmonic) sound to irregular noise. Higher HNR = cleaner voice.
 - **CPP (Cepstral Peak Prominence)** – Another measure related to how strongly harmonic and periodic the voice is.
 - **LTAS (Long-Term Average Spectrum)** – The average energy at each frequency across a whole utterance.
-

5. How It Works – From Audio File to Feature Table

Imagine we have one audio file: `speaker12_sentence3_trustworthy.wav`.

Step 1 – Load the waveform

- Software reads the file and gets an array of numbers (samples).
- Example: `[0.01, 0.02, 0.01, -0.01, ...]` with 48,000 samples per second.

Step 2 – Divide into frames

- Split the signal into overlapping windows, e.g., 30 ms each.
- For each frame, we compute basic quantities:
 - Instantaneous pitch
 - Instantaneous loudness
 - Jitter, shimmer, etc.

Step 3 – Aggregate across frames

- For each utterance, we compute **summary statistics**:
 - Mean F0 across all frames (average pitch)
 - SD F0 (how much F0 changes)

- Minimum and maximum F0
- Mean HNR, mean shimmer, mean CPP
- Duration of the utterance

Step 4 – Save as a row in a table

file_name	mean_F0	sd_F0	HNR	shimmer	CPP	duration	intent
speaker12_sentence3_trustworthy.wav	195.2	21.5	11.2	0.32	15.1	1.73	1

Step 5 – Repeat for all files

- Do this for all 1,152 files, and you get a big table — the **feature matrix**.
- This matrix becomes the input to machine learning models.

In this course, we'll mostly work with **already-extracted feature tables** (CSV files) to make things easier. But later we'll discuss tools like **Praat**, **VoiceLab**, or Python libraries (e.g., **librosa**, **parselmouth**) that can compute features from raw audio.

6. Practice Exercises

Exercise 1 – Matching Concept to Number For each concept, choose the most relevant feature:

a) How high or low the voice sounds overall. b) How clean vs noisy the voice is. c) How shaky the pitch is. d) How long the person talks.

Features: **mean_F0**, **HNR**, **jitter**, **duration**.

Exercise 2 – Reading a Feature Table

Suppose we have this small table:

Utterance	mean_F0 (Hz)	HNR (dB)	duration (s)	intent_label
A	170	9.0	1.8	0
B	195	11.2	1.6	1

a) Which utterance has the **higher pitch**? b) Which utterance has the **cleaner voice quality**? c) Which one is labeled as trustworthy?

Exercise 3 – Why Use Multiple Features?

Explain in 3–4 sentences why it might be **better to use several features together (pitch, HNR, CPP, etc.) instead of only one** when trying to detect trustworthy intent.

Exercise 4 – Designing a New Feature

Imagine you want a feature called **speech_rate** that measures **how fast someone is speaking** (syllables per second).

a) Which raw measurements might you need (e.g., number of syllables, duration)? b) How would you roughly compute **speech_rate**?

Exercise 5 – Thinking About Limitations

Is it possible for two people to have **very different voices** but produce **similar feature values** (mean F0, HNR, etc.)? In 2–3 sentences, discuss what this might mean for a machine learning model trained only on acoustic features.

7. Solutions / Model Answers

Exercise 1 – Answer

a) How high or low the voice sounds → **mean_F0** b) How clean vs noisy the voice is → **HNR** c) How shaky the pitch is → **jitter** d) How long the person talks → **duration**

Exercise 2 – Answer

a) Higher pitch → **Utterance B** (195 Hz vs 170 Hz) b) Cleaner voice (higher HNR) → **Utterance B** (11.2 dB vs 9.0 dB) c) Trustworthy utterance → **Utterance B** (intent_label = 1)

Exercise 3 – Sample Answer

Trustworthiness is a complex perception. Pitch alone might sometimes correlate with sounding friendly or confident, but not always. HNR, jitter, shimmer, CPP, and duration all capture different aspects of how the voice sounds. By using many features together, the model can notice patterns like “slightly higher pitch + stable voice + clear harmonics,” which may be more predictive than any single feature alone.

Exercise 4 – Sample Answer

a) You would need at least:

- The **number of syllables** in the utterance.
- The **duration** of the utterance in seconds.

b) You could compute:

$$[\text{speech_rate}] = \frac{\text{number_of_syllables}}{\text{duration_seconds}}$$

This gives the average number of syllables spoken per second.

Exercise 5 – Sample Answer

Yes, two different people might have similar mean pitch, HNR, and duration values even if their voices sound different to us. This means the model might not be able to tell them apart based only on these features. It reminds us that acoustic features are a simplified representation of a very rich signal and that models can miss important cues that humans use.

8. Q&A – Common Student Questions

1. **Q:** Why do we split audio into frames instead of analyzing the whole file at once? **A:** Because many features (like pitch) change over time. Frames let us measure how features behave locally, then we summarize them (mean, SD, etc.) for the whole utterance.

2. **Q:** Are jitter and shimmer always bad? **A:** Not exactly. High jitter/shimmer can indicate instability or strain, but some natural variation is normal. It depends on context and how much they deviate from typical values.
3. **Q:** Why does HNR matter for trust? **A:** Higher HNR means a clearer, more periodic voice, which might sound healthier and more controlled. People might unconsciously associate this with confidence and reliability.
4. **Q:** What's the difference between HNR and CPP? **A:** Both relate to how harmonic/periodic the voice is, but they're computed differently. HNR looks at energy ratios, while CPP looks at peaks in the **cepstrum** (a transform of the spectrum). For this course, just remember they both measure voice clarity/periodicity.
5. **Q:** Why do we need SD F0 if we already know mean F0? **A:** Mean F0 says where the pitch is centered; SD F0 tells us how much it **moves around**. Two speakers could have the same average pitch but very different pitch variation.
6. **Q:** Are these features specific to English? **A:** No. Pitch, HNR, jitter, shimmer, CPP, etc., are properties of human voice production, not of a specific language. They can be measured in any spoken language.
7. **Q:** Can we use deep learning to automatically learn features instead? **A:** Yes. Deep models (like CNNs on spectrograms) can learn their own features from raw audio. But hand-crafted features are still very useful, easier to interpret, and require less data.
8. **Q:** Do we need to understand the exact math formulas of these features to use them? **A:** For this project, no. It's more important to know **what they represent** conceptually and how they might relate to sounding trustworthy.
9. **Q:** Are features noisy or perfect? **A:** They can be noisy. Background noise, recording quality, and errors in pitch detection all affect feature values. That's why we often use robust statistics and cross-validation.
10. **Q:** What if two features are strongly correlated? **A:** It means they carry similar information (e.g., HNR and CPP might both reflect voice periodicity). Models like Random Forest can handle this fine; for linear models, we may need to be more careful, but it's not a disaster.

9. Quiz (10 Questions + Answers)

Q1. What does F0 represent? a) Loudness b) Pitch c) Duration d) Noise level

Answer: b) Pitch.

Q2. Which feature measures tiny random fluctuations in **pitch**? **Answer:** Jitter.

Q3. Which feature measures tiny random fluctuations in **loudness**? **Answer:** Shimmer.

Q4. What does a higher HNR generally indicate about the voice? a) More noise b) Less noise and clearer harmonics c) Longer duration d) Higher speech rate

Answer: b) Less noise and clearer harmonics.

Q5. True or False: Duration is an acoustic feature. **Answer:** True.

Q6. What is a "feature vector"? **Answer:** A list of feature values describing one example (one utterance), e.g., `[mean_F0, HNR, duration, ...]`.

Q7. SD F0 measures: a) The average loudness b) The average pitch c) How much the pitch varies over time d) How long the speech lasts

Answer: c) How much the pitch varies over time.

Q8. Why might LTAS be useful? **Answer (short):** It summarizes how energy is distributed across frequencies (low vs high), which can reflect voice timbre and quality (e.g., bright vs dark voice).

Q9. If trustworthy intent tends to have **higher mean F0** and **higher HNR** in a dataset, what might that suggest?

Answer: That in this dataset, voices that sound more trustworthy are modeled as slightly higher-pitched and clearer (less noisy) compared to neutral voices.

Q10. True or False: Once you have the feature table, you no longer need the audio files for machine learning. **Answer:** False – you don't *need* them to run the same ML model, but they are still important for checking quality, extracting new features, or running different types of audio-based models.

10. Mini Practice Project – Exploring Acoustic Features (with .zip)

Mini Project Title: *Exploring Synthetic Acoustic Features for Neutral vs Trustworthy Speech*

Goal: Give the student a **small, friendly dataset** of acoustic features and let them:

- Inspect the feature table.
- Compare averages for neutral vs trustworthy intent.
- See how features differ between the two groups.

I've created a ready-to-use project folder with:

- `synthetic_acoustic_features.csv` – a small synthetic dataset (60 rows) with:
 - `mean_f0_hz` – mean pitch (Hz)
 - `sd_f0_hz` – pitch variability
 - `hnr_db` – Harmonics-to-Noise Ratio
 - `shimmer_db` – shimmer in dB
 - `cpp_db` – Cepstral Peak Prominence in dB
 - `duration_seconds` – utterance length in seconds
 - `intent_label` – 0 = neutral, 1 = trustworthy
- `analyze_acoustic_features.py` – a **fully commented** script that:
 - Loads the dataset
 - Prints the first rows and the shape
 - Shows basic statistics (`describe()`)
 - Computes mean feature values for each intent
 - Computes differences: (trustworthy – neutral)
 - Prints a correlation matrix for the features
 - Gives a short interpretation hint
- `README.txt` – explains how to install and run, and what to look for.

How the student should use it:

1. Unzip the file to a folder.
2. Open a terminal in the `lesson2_acoustic_features_intro` folder.
3. Install dependencies (once):

```
pip install pandas numpy
```

4. Run the script:

```
python analyze_acoustic_features.py
```

5. Carefully read:

- The mean feature values for neutral vs trustworthy.
- The difference in means.
- The correlation matrix.

Mini Project Extension Ideas (Optional):

- Ask the student to:
 - Identify **which features change the most** between neutral and trustworthy.
 - Write a short explanation of **why** those features might matter.
 - Modify one feature column slightly (e.g., add 10 Hz to `mean_f0_hz` for trustworthy) and re-run to see how group means change.

11. References

For building intuition about acoustic features:

- Introductory articles on:
 - "Fundamental Frequency (F0) in Speech"
 - "Harmonics-to-Noise Ratio in Voice Analysis"
 - "Jitter and Shimmer in Voice Quality"
- Tutorials using:
 - **Praat** (popular tool for phonetics): lots of online guides on measuring F0, jitter, shimmer.
 - **librosa** (Python audio library): introductory tutorials on reading audio and computing features.
- Any high-school-friendly videos on:
 - "What is a spectrogram?"
 - "Basic features of speech signals."

(These don't have to be specific links yet; we'll introduce more concrete tools/resources in later lessons.)

12. Additional Information / Teacher Tips

- **Use their own speech** You can have the student record 2–3 short sentences and talk about how features might differ between their “normal” voice and their “trustworthy” voice, even before computing anything.
 - **Link to the paper’s tables** Show them one of the feature tables from the paper and explain that **our synthetic dataset is a simplified version** of that. The goal is to make the real paper’s tables feel less scary.
 - **Connect to math gently** When you mention “mean” or “standard deviation,” tie it to what they already know from statistics: average vs spread. No heavy formulas needed.
-

Lesson 3 – Understanding the Trust Dataset: Speakers, Sentences & Labels

1. Lesson Summary

In this lesson, the student will learn **how the real “trustworthy intent” dataset is organized**:

- What counts as **one data point** (an utterance).
- How **speakers, sentences, and intents** (neutral vs trustworthy) are stored.
- What **metadata** is included (age group, sex, ethnicity).
- How this structure supports **fair machine learning**, where we can check performance for different demographic groups.

We’ll also connect this to **practical skills**: reading a CSV with pandas, exploring rows/columns, counting speakers, and checking class balance. The student will practice this with a small synthetic dataset that mimics the structure of the real one.

2. Key Points

- The real dataset contains **1,152 utterances from 96 speakers**. Each utterance is one short spoken sentence.
 - Speakers come from **three ethnic backgrounds** (white, Black, South Asian), **two age groups** (younger 18–45, older 60+), and both **sexes**.
 - Each speaker recorded sentences in **two intents**:
 - Neutral (no special intent)
 - Trustworthy intent (trying to sound trustworthy)
 - The data table is organized with:
 - **One row per utterance** (sentence spoken once)
 - Columns for **speaker metadata** (ID, age group, sex, ethnicity)
 - Columns for **experimental condition** (neutral vs trustworthy)
 - Columns for **acoustic features** (pitch, HNR, etc.)
 - This structure lets us:
 - Count how many examples per class (neutral/trustworthy)
 - Check how examples are distributed across demographics
 - Apply **cross-validation** where we hold out speakers (not just random rows).
 - Understanding the dataset structure is essential before training any model.
-

3. Real-World Examples or Stories

1. **Class roster analogy** Imagine a spreadsheet listing all your classmates. Each row is one **student**; columns: name, age, grade, club membership. Here the dataset is similar, but each row is one **utterance**: who spoke it, how, and in what condition.
2. **Survey data** When you fill out a survey (age, gender, answers to questions), the results end up in a table. In this dataset, each utterance is like one survey response, with the **voice features** and **intent label** as answers.
3. **Sports stats** In a basketball stats table, each row might be one game: player, opponent, points, rebounds, etc. For this project, each row is one spoken sentence: speaker, intent, duration, pitch, etc.
4. **Fairness in grading** If a teacher only looked at a few students' tests, they might accidentally bias the results. Similarly, if our dataset had mostly young white male speakers, our model might not generalize to others. That's why the paper carefully balances demographics.

4. Terminology Explained

- **Observation / sample / example** – One row in the dataset; here, one utterance.
 - **Metadata** – Extra information about each example that is not the main target (e.g., speaker ID, age group).
 - **Label** – The target value we want to predict (here: neutral vs trustworthy intent).
 - **Class** – One category of the label (neutral or trustworthy).
 - **Class balance** – How many examples belong to each class. A balanced dataset has similar counts.
 - **Speaker ID** – A code (e.g., S01, S02) identifying each speaker.
 - **Within-subject design** – Each speaker appears in both conditions (neutral and trustworthy); we compare each person to themselves.
 - **Train / test split** – Dividing data into a part for training the model and a part for evaluating it.
 - **Speaker-independent evaluation** – Making sure that speakers in the training set are **not** in the test set, to test generalization to new people.
-

5. How It Works – Dataset Structure in Practice

Let's imagine a simplified version of the real trust dataset table.

One row per utterance Example row:

speaker_id	age_group	sex	ethnicity	sentence_id	intent_label	duration	mean_F0	HNR	...
S12	younger	male	south_asian	3	1	1.65	193.4	11.0	...

- **speaker_id** – which person spoke
- **age_group** – younger or older
- **sex** – male or female
- **ethnicity** – white / Black / South Asian
- **sentence_id** – which sentence prompt they read (like "You can count on me")
- **intent_label** – 0 = neutral, 1 = trustworthy
- Acoustic features follow: **duration**, **mean_F0**, **HNR**, etc.

Within each speaker:

- They read multiple sentences.
- Each sentence is recorded twice: **neutral** and **trustworthy intent**.

- That means for each speaker, there are multiple rows with `intent_label = 0` and multiple with `intent_label = 1`.

Why this structure is important:

- We can **group by** `speaker_id` to see how many utterances each speaker has.
- We can check that **each speaker appears in both classes**, which is crucial for the experimental design.
- We can group by demographic columns (e.g., `ethnicity`) to see if the dataset is balanced across groups.
- For fair evaluation, we can perform **leave-one-speaker-out cross-validation**:
 - Train on 95 speakers, test on the 1 left-out speaker
 - Repeat for every speaker This tests whether the model generalizes to **new voices**.

In this lesson's mini project, the student will work with a small synthetic dataset that has the same **logical structure**, so they can practice with pandas before touching the full dataset.

6. Practice Exercises

Exercise 1 – Identify the Observation

In the trust dataset, what is considered **one observation** (one row)? a) One speaker b) One sentence text (like "You can count on me") c) One utterance (a sentence spoken once with a particular intent) d) One feature like pitch

Exercise 2 – Matching Columns

Match each concept to the most likely column name:

1. Whether the speaker tried to sound trustworthy
2. Which demographic group the speaker belongs to (e.g., 18–45 or 60+)
3. Which person spoke the sentence
4. The average pitch of that utterance

Columns: `speaker_id`, `age_group`, `intent_label`, `mean_f0_hz`

Exercise 3 – Why Metadata Matters

Explain in 3–4 sentences why it is important to store metadata like `age_group`, `sex`, and `ethnicity` when building a dataset for trust in voices.

Exercise 4 – Counting Examples

Suppose you load the dataset and compute:

```
data["intent_label"].value_counts()
```

The output is:

```
0    580
1    572
Name: intent_label, dtype: int64
```

a) Which label is more frequent? b) Is the dataset roughly balanced? Why or why not?

Exercise 5 – Speaker-Independent Splits

Why is it a bad idea to randomly split rows for train/test if some rows from the **same speaker** end up in both sets? Explain in 2–3 sentences.

7. Solutions / Model Answers

Exercise 1 – Answer

Correct answer: **c) One utterance** Each row in the dataset represents one specific recording of a sentence spoken once under a particular intent.

Exercise 2 – Answer

1 → **intent_label** 2 → **age_group** 3 → **speaker_id** 4 → **mean_f0_hz**

Exercise 3 – Sample Answer

Metadata like age group, sex, and ethnicity tells us **who** the voices belong to. This helps ensure that the dataset is not dominated by only one group, which could make the model biased. It also allows researchers to check whether the model performs equally well for different groups, which is important for fairness and generalization in real-world applications.

Exercise 4 – Answer

a) Label **0** (neutral) is slightly more frequent (580 vs 572). b) Yes, it's roughly balanced because the numbers are very close. There is no huge difference between the counts, so the model will see a similar number of examples from each class.

Exercise 5 – Sample Answer

If rows from the same speaker appear in both train and test sets, the model might “cheat” by learning **speaker-specific patterns** instead of general patterns about trust. It could perform very well on the test set simply because it recognizes the speaker's voice, not because it truly understands trustworthy intent. That's why we prefer speaker-independent splits like leave-one-speaker-out.

8. Q&A – Common Student Questions

- Q:** Why not store one row per speaker instead of one row per utterance? **A:** Because each speaker produces multiple utterances under different conditions. We need separate rows to capture each specific recording and its features.

2. **Q:** Can a dataset have multiple labels? **A:** Yes. For example, you could have `intent_label` (neutral vs trustworthy) and `emotion_label` (happy, sad, etc.). In this study, the main label is intent.
 3. **Q:** What is the difference between `speaker_id` and `sentence_id`? **A:** `speaker_id` tells you **who** spoke; `sentence_id` tells you **which sentence** they were reading.
 4. **Q:** Why is class balance important? **A:** If one class is much larger, a model might just guess that class all the time and still get high accuracy, which is misleading.
 5. **Q:** What is “within-subject” design? **A:** It means each participant (speaker) experiences all conditions (neutral and trustworthy), so we can compare how each person changes their voice between conditions.
 6. **Q:** Could we treat each speaker as a separate dataset? **A:** You could analyze each speaker individually, but for machine learning you usually want to combine them to learn general patterns while still keeping track of who is who.
 7. **Q:** Why are demographic columns not used directly as features for trust classification? **A:** The goal is to detect trust from **how the voice sounds**, not from who the person is. Demographic info is mainly for analyzing fairness and dataset diversity, not for prediction.
 8. **Q:** How do we represent text content in this dataset? **A:** In this paper, the focus is on **acoustic features**, not text. The sentence ID indicates which prompt was used, but we mostly ignore the actual words.
 9. **Q:** Is it okay if some speakers have a few more utterances than others? **A:** Slight differences are okay, but very unbalanced speakers could bias the model. The paper’s design aims for relatively consistent counts per speaker.
 10. **Q:** Can we add more metadata later? **A:** Yes, as long as you can link it to existing rows via `speaker_id` or a file name. For example, you could later add hearing test results or personality measures if available.
-

9. Quiz (10 Questions + Answers)

Q1. In this dataset, what does each row represent? a) One speaker b) One acoustic feature c) One utterance (sentence spoken once) d) One demographic group

Answer: c) One utterance.

Q2. Which column is most likely the **label** for our classification task? a) `speaker_id` b) `intent_label` c) `age_group` d) `mean_f0_hz`

Answer: b) `intent_label`.

Q3. What type of information is `age_group`? a) Feature used for prediction b) Label c) Metadata d) Noise

Answer: c) Metadata (though it *could* be used as a feature for analysis, its main role is contextual info).

Q4. True or False: The dataset includes both neutral and trustworthy recordings for each speaker.

Answer: True (that’s part of the within-subject design).

Q5. Why do we care about how many examples each class has? **Answer:** Because a highly imbalanced dataset can lead to biased models and misleading accuracy; balanced classes make evaluation more meaningful.

Q6. What does `speaker_id` help us do? a) Measure pitch b) Identify which group a sample belongs to c) Track which utterances belong to the same speaker d) Compute duration

Answer: c) Track which utterances belong to the same speaker.

Q7. Which of the following is **not** metadata? a) `speaker_id` b) `age_group` c) `intent_label` d) `ethnicity`

Answer: c) `intent_label` – that's the target label.

Q8. Why is it good that the dataset includes different ethnicities and age groups? **Answer:** It reduces bias and makes the results more generalizable across diverse speakers.

Q9. True or False: When evaluating the model, it's better to keep all utterances from a single speaker either in train or test, but not both.

Answer: True – this is speaker-independent evaluation.

Q10. If we see that one demographic group has very few examples, what might that indicate? **Answer:** The dataset may be unbalanced across demographics, and we should be careful about drawing conclusions for that group; the model may not perform well for them.

10. Mini Practice Project – Synthetic Trust Dataset Structure (with .zip)

Mini Project Title: *Exploring a Synthetic Trust Dataset Structure with Pandas*

Goal: Let the student practice **dataset exploration** on a small synthetic table that mimics the structure of the full trust dataset.

I've created a project folder that includes:

- `synthetic_trust_dataset_structure.csv`
 - Columns:
 - `speaker_id` – e.g., S01, S02, ...
 - `age_group` – "younger" or "older"
 - `sex` – "female" or "male"
 - `ethnicity` – "white", "black", "south_asian"
 - `sentence_id` – integer ID of the sentence per speaker
 - `intent_label` – 0 = neutral, 1 = trustworthy
 - `duration_seconds` – utterance duration
 - `mean_f0_hz` – mean pitch
- `explore_trust_dataset_structure.py`
 - Fully commented, doing:
 - Load CSV with pandas
 - Print head, shape, column names
 - Count utterances per speaker
 - Check overall class distribution (`intent_label`)
 - Group by `speaker_id` and `intent_label` to verify each speaker has both intents

- Show how speakers are distributed across `age_group`, `sex`, `ethnicity`
 - Show summary stats for `duration_seconds` and `mean_f0_hz`
 - Provide interpretation hints
- `README.txt` with step-by-step instructions.

How the student should use it:

1. Unzip the file.
2. Open a terminal in `lesson3_trust_dataset_eda`.
3. Install pandas (if not already):

```
pip install pandas
```

4. Run:

```
python explore_trust_dataset_structure.py
```

5. Read the printed results carefully and answer:
 - How many speakers are there?
 - How many utterances per speaker?
 - How many neutral vs trustworthy utterances overall?
 - Do all speakers have both intents?

This gives them a **hands-on mental model** of how the real trust dataset is structured.

11. References

- The trust dataset paper (for structure & design details):
 - *Human voices communicating trustworthy intent: A demographically diverse speech audio dataset* – Scientific Data (2025).
 - Basic pandas tutorials for high school students:
 - Intro to `DataFrame`, `.head()`, `.shape`, `.describe()`, `.groupby()`, and `value_counts()`.
 - Simple dataset/EDA resources:
 - “What is a dataset?” articles and videos that explain rows, columns, and metadata.
-

12. Additional Information / Teacher Tips

- **Have them draw the table** Ask the student to draw on paper a small mock-up: 3 speakers × 4 utterances, with columns for metadata and labels. Visualizing helps a lot.
- **Connect to fairness early** You can already ask: “How would we notice if there were only young speakers?” or “Why is that a problem?” This sets up later discussions on bias and fairness in AI.

- **Build habits for EDA** Encourage them to always start a new project with:

- `df.head()`
- `df.shape`
- `df.dtypes`
- `df.describe()`
- `df['label'].value_counts()`

Lesson 4 – Exploratory Data Analysis (EDA) of Trustworthy Voices

1. Lesson Summary

In this lesson, the student will learn how to **explore the trust dataset with data science tools** before training any model.

We focus on **Exploratory Data Analysis (EDA)**:

- Checking **class balance** (how many neutral vs trustworthy utterances).
- Looking at **basic statistics** of key features (mean, std, min, max).
- Comparing **feature distributions** between neutral and trustworthy speech.
- Creating **simple histograms** to visualize differences (e.g., pitch & HNR).

The goal: build intuition like

“Trustworthy utterances tend to have slightly higher pitch and HNR in this dataset,”

and to form good habits: **always inspect your data first**.

2. Key Points

- EDA is the process of **getting to know your data** before modeling.
- Always check:
 - **Shape** of the dataset (rows, columns).
 - **Column names** and data types.
 - **Missing values** and obvious errors.
 - **Class balance** for the target label.
- Use **summary statistics** (`describe()` in pandas) to see:
 - Min, max, mean, standard deviation of features.
- Compare features across **groups** (here: neutral vs trustworthy; optionally across age/sex/ethnicity).
- Visual tools like **histograms and boxplots** help see:
 - Are trustworthy utterances shifted toward higher pitch?
 - Is HNR generally higher for trustworthy speech?
- EDA often reveals:
 - Outliers
 - Data entry problems

- Potential biases
 - Good EDA makes your later machine learning results **more trustworthy and interpretable**.
-

3. Real-World Examples or Stories

1. **Doctor checking your vitals** Before suggesting any treatment, a doctor checks your **basic stats**: temperature, pulse, blood pressure. EDA is like checking the dataset's vitals.
 2. **Coach checking team stats** A basketball coach looks at points, rebounds, and shooting percentages for each player. They might see that one player always scores more in home games. Similarly, we might see that trustworthy utterances have higher average pitch.
 3. **Quality control in a factory** Each product is measured; EDA is like checking the distribution of product sizes. If some are too big or too small, you know something is wrong. In our data, if durations are negative... we know something is wrong too!
 4. **First impression of a new class** On the first day, you get a sense of how many students there are, how many like math, etc. EDA is your "first impression" of a dataset.
-

4. Terminology Explained

- **EDA (Exploratory Data Analysis)** – The process of exploring a dataset to understand its main characteristics, usually using summary statistics and plots.
 - **Summary statistics** – Numbers that summarize a feature: mean, median, min, max, standard deviation.
 - **Distribution** – Describes how a feature's values are spread out (e.g., centered at 180 Hz, most values between 150–210 Hz).
 - **Histogram** – A bar chart that shows how many values fall into different ranges (bins).
 - **Boxplot** – A plot showing the median, quartiles, and potential outliers of a feature.
 - **Outlier** – A data point that is very different from most others (e.g., mean pitch of 500 Hz when others are around 150–250 Hz).
 - **Class balance** – How evenly the dataset is split between labels (here, neutral vs trustworthy).
 - **Groupby** – A pandas operation to compute statistics separately for different groups (e.g., by label or by age group).
-

5. How It Works – EDA Step-by-Step

Suppose you load the feature table for the trust dataset (or a synthetic version).

Step 1 – Look at the basic info

```
import pandas as pd

data = pd.read_csv("trust_features.csv")
print(data.head())
print(data.shape)
print(data.dtypes)
```

- `head()` shows the first few rows.
- `shape` shows how many rows and columns.

- `dtypes` shows whether columns are numbers, strings, etc.

Step 2 – Check label distribution

```
print(data["intent_label"].value_counts())
```

- Are neutral and trustworthy counts similar?
- If one class is much bigger, we note this for later.

Step 3 – Summary statistics for numeric features

```
numeric_cols = ["duration_seconds", "mean_f0_hz", "sd_f0_hz", "hnr_db"]  
print(data[numeric_cols].describe())
```

- We see min, max, mean, standard deviation, quartiles.
- Check if any values look impossible (e.g., negative duration).

Step 4 – Compare neutral vs trustworthy averages

```
print(data.groupby("intent_label")[numeric_cols].mean())
```

- We can see if, on average, trustworthy utterances have:
 - Higher `mean_f0_hz`
 - Higher `hnr_db`
 - Different `duration_seconds`

This mimics the analysis in the paper where certain acoustic features are higher/lower for trustworthy intent.

Step 5 – Visualize distributions

```
import matplotlib.pyplot as plt  
  
neutral = data[data["intent_label"] == 0]  
trustworthy = data[data["intent_label"] == 1]  
  
plt.figure()  
plt.hist(neutral["mean_f0_hz"], bins=20, alpha=0.7, label="Neutral")  
plt.hist(trustworthy["mean_f0_hz"], bins=20, alpha=0.7, label="Trustworthy")  
plt.title("Histogram of mean_f0_hz by intent")  
plt.xlabel("mean_f0_hz (Hz)")  
plt.ylabel("Count")  
plt.legend()  
plt.show()
```

- If the “trustworthy” histogram peaks at slightly higher values, we see that trustworthy speech tends to be higher-pitched in this dataset.

Step 6 – Reflect & hypothesize

- Based on EDA, we form hypotheses:
 - "I expect a classifier that uses mean pitch & HNR might perform better than random."
 - "I should be careful about speaker imbalance or potential outliers."

These insights guide how we design and interpret our models in later lessons.

6. Practice Exercises

Exercise 1 – Reading Class Distribution

You run:

```
data["intent_label"].value_counts()
```

and get:

```
0    180
1    180
Name: intent_label, dtype: int64
```

a) How many total utterances are in the dataset? b) Is the dataset balanced? Explain.

Exercise 2 – Interpreting `describe()`

For `mean_f0_hz`, `data["mean_f0_hz"].describe()` prints:

```
count    360.000000
mean     182.500000
std       22.000000
min      120.000000
25%      166.000000
50%      180.000000
75%      198.000000
max      240.000000
```

a) What is the average (mean) pitch? b) What pitch value is at the 50% (median) mark? c) Are there any obviously impossible values?

Exercise 3 – Group Means

You compute:

```
data.groupby("intent_label")[["mean_f0_hz", "hnr_db"]].mean()
```

and get:

intent_label	mean_f0_hz	hnr_db
0	175.0	9.7
1	190.0	11.0

Describe in words how neutral and trustworthy utterances differ based on these averages.

Exercise 4 – Histogram Interpretation

You see a histogram of `mean_f0_hz` where:

- The neutral distribution mostly peaks around 170–180 Hz.
- The trustworthy distribution peaks around 185–195 Hz and is slightly shifted to the right.

What does this tell you about pitch differences between the two intents?

Exercise 5 – EDA & Outliers

Suppose in the summary stats you notice one utterance with `duration_seconds = 15.0`, while most are around 1.5–2.0 seconds.

a) Why might this be an outlier? b) What steps might you take to handle it?

7. Solutions / Model Answers

Exercise 1 – Answer

a) Total utterances = 180 + 180 = **360**. b) Yes, it's balanced; there are equal numbers of neutral (0) and trustworthy (1) utterances.

Exercise 2 – Answer

a) Average (mean) pitch = **182.5 Hz**. b) The median (50%) pitch = **180 Hz**. c) Values range from 120 Hz to 240 Hz, which are realistic for typical adult speech, so there are no obviously impossible values.

Exercise 3 – Answer

Neutral utterances (label 0) have an average pitch of 175 Hz and HNR of 9.7 dB. Trustworthy utterances (label 1) have a higher average pitch (190 Hz) and higher HNR (11.0 dB). So, in this dataset, trustworthy speech tends to be **higher-pitched and clearer** (less noisy) than neutral speech.

Exercise 4 – Answer

The trustworthy distribution being shifted to the right means that, overall, trustworthy utterances tend to have **higher pitch values** than neutral ones. The overlap shows they are not completely separate, but there is a noticeable trend toward higher pitch for trustworthy intent.

Exercise 5 – Sample Answer

a) It's an outlier because almost all utterances are around 1.5–2.0 seconds, and 15 seconds is much longer than the rest. It might be a recording or segmentation error. b) You could:

- Double-check the original audio to see if it's a valid sample.
- If it's clearly incorrect, remove it from the dataset.
- Or cap extremely long durations if they result from rare glitches.

8. Q&A – Common Student Questions

1. **Q:** Why do we do EDA before training models? **A:** To understand the data, catch errors, see patterns, and avoid building models on bad or misunderstood data.
2. **Q:** What if the class distribution is very imbalanced? **A:** We might need techniques like resampling, different metrics (e.g., F1, AUC), or adjusting decision thresholds. But first, we must **notice** the imbalance through EDA.
3. **Q:** Is looking at `describe()` enough? **A:** It's a good start, but visualizing distributions (histograms, boxplots) and checking relationships between features is also important.
4. **Q:** Why use histograms instead of just means? **A:** Means can hide details (like multi-modal distributions). Histograms show how the values are spread out, not just the average.
5. **Q:** What is a good number of bins for a histogram? **A:** There's no single perfect answer; 10–30 bins is common for medium-sized datasets. The key is to see a clear shape without too much noise.
6. **Q:** Can we use boxplots instead of histograms? **A:** Yes! Boxplots are great for comparing medians and spread across groups and spotting outliers quickly.
7. **Q:** Should we remove all outliers? **A:** Not automatically. Some outliers are real and meaningful. We should investigate them and decide based on domain knowledge.
8. **Q:** Do we need to normalize features during EDA? **A:** For basic EDA, we usually look at raw values. Normalization is more important when training certain models (e.g., logistic regression, neural networks).
9. **Q:** How do we check relationships between two features? **A:** We can use scatter plots and correlation matrices to see how they move together.
10. **Q:** Does EDA change the dataset? **A:** EDA itself is just viewing/analyzing. But based on EDA, we might then clean or transform the dataset (e.g., removing invalid rows), which *does* change it.

9. Quiz (10 Questions + Answers)

Q1. EDA stands for: a) Exploratory Data Analysis b) Experimental Data Analytics c) Extended Data Aggregation d) Exploratory Distribution Assessment

Answer: a) Exploratory Data Analysis.

Q2. Which of the following is *not* a typical EDA step? a) Checking class balance b) Computing summary statistics c) Training a deep neural network d) Plotting histograms

Answer: c) Training a deep neural network.

Q3. `data.shape` returns `(360, 12)`. What does 360 represent? **Answer:** The number of rows (utterances) in the dataset.

Q4. A histogram is best used to: a) Show the exact median value b) Show the distribution of a single numeric feature c) Show relationships between many features d) Show text data

Answer: b) Show the distribution of a single numeric feature.

Q5. If `data["intent_label"].value_counts()` returns `0: 300, 1: 60`, is the dataset balanced? **Answer:** No, it is strongly imbalanced; neutral examples are much more frequent than trustworthy ones.

Q6. What does `data[numeric_cols].describe()` not show? a) Mean b) Standard deviation c) Median d) Exact histogram

Answer: d) Exact histogram.

Q7. If trustworthy utterances have higher mean `hnr_db` than neutral ones, what does that suggest? **Answer:** Trustworthy utterances might have **clearer, less noisy** voices in this dataset.

Q8. True or False: Seeing an impossible value (like negative duration) means we should ignore it and move on.

Answer: False – we should investigate and likely fix or remove it.

Q9. Which pandas method is most useful for computing means per class? a) `.head()` b) `.groupby()` c) `.merge()` d) `.drop()`

Answer: b) `.groupby()`.

Q10. Why might we create separate histograms for neutral and trustworthy utterances? **Answer:** To visually compare how feature distributions differ between the two classes.

10. Mini Practice Project – EDA & Simple Plots (Downloadable .zip)

Mini Project Title: *EDA and Histograms for Neutral vs Trustworthy Speech*

I've prepared a mini project folder that includes:

- `synthetic_trust_eda_dataset.csv`
 - Each row = one utterance, with:
 - `speaker_id`
 - `age_group`
 - `sex`
 - `ethnicity`
 - `sentence_id`
 - `intent_label` (0 = neutral, 1 = trustworthy)
 - `duration_seconds`
 - `mean_f0_hz`

- `sd_f0_hz`
- `hnr_db`
- `trust_eda_plots.py`
 - Fully commented script that:
 - Loads the dataset
 - Shows head, shape, columns
 - Prints class distribution
 - Prints summary stats for numeric features
 - Computes mean feature values by intent
 - Plots **two histograms**:
 - `mean_f0_hz` for neutral vs trustworthy
 - `hnr_db` for neutral vs trustworthy
 - Prints interpretation hints
- `README.txt` with run instructions.

How the student should use it:

1. Unzip the file.
2. Open a terminal / command prompt in `lesson4_trust_eda_plots`.
3. Install dependencies (if needed):

```
pip install pandas matplotlib
```

4. Run:

```
python trust_eda_plots.py
```

5. Two histogram windows will appear:
 - Compare the neutral vs trustworthy distributions for `mean_f0_hz` and `hnr_db`.
6. Answer questions like:
 - Are trustworthy utterances shifted toward higher pitch?
 - Are HNR values typically higher for trustworthy speech?

This mini project directly practices **EDA + simple plotting**, exactly what you'll do on the real dataset later.

11. References

- Any introductory EDA resources with pandas:

- "Pandas `.head()`, `.describe()`, `.groupby()`, and `value_counts()`" tutorials.
- Matplotlib beginner guides:
 - "Creating histograms with matplotlib" tutorial.
- Optional deeper reading:
 - Articles or videos on:
 - "Understanding data distributions"
 - "Introduction to outliers and boxplots"

12. Additional Information / Teacher Tips

- **Encourage curiosity** Let the student ask their own questions: "Do older speakers show the same pattern?" "What if we separate by sex or ethnicity?" Even if you don't answer all of them now, this builds research thinking.
- **Teach reproducibility habits** Suggest that the student always keeps their EDA code in a separate script or notebook so they can re-run it when the dataset changes.
- **Link to later lessons** Explain that this EDA will make it easier to:
 - Understand feature importance later.
 - Interpret why the Random Forest or Logistic Regression model performs the way it does.

Lesson 5 – Supervised Learning & Baseline Models (Logistic Regression + Random Forest)

1. Lesson Summary

In this lesson, the student will connect everything learned so far (features, labels, dataset structure, EDA) into a **complete supervised learning pipeline**.

We'll focus on:

- What **supervised learning** is (using labeled data to learn patterns).
- What a **binary classifier** does (predict neutral vs trustworthy).
- How to:
 - Split data into **train** and **test** sets.
 - Train **Logistic Regression** and **Random Forest** models.
 - Evaluate them using **accuracy, precision, recall, and F1-score**.
- Understand these models as **baseline classifiers** that approximate what the paper did:
 - Logistic Regression: simple, linear, interpretable.
 - Random Forest: ensemble of decision trees capturing nonlinear patterns.

By the end, the student will know how to **train and evaluate simple models in Python** using scikit-learn on a trust-like acoustic feature dataset.

2. Key Points

- **Supervised learning** uses input features (X) and known labels (y) to learn a mapping from $X \rightarrow y$.
 - Our problem is **binary classification**:
 - 0 = neutral intent
 - 1 = trustworthy intent
 - The workflow:
 1. Load features and labels from a table.
 2. Split into **train** and **test** sets.
 3. Train a model on the train set.
 4. Evaluate on the test set using metrics.
 - **Train set**: used for fitting the model.
 - **Test set**: used only to evaluate how the model performs on unseen data.
 - **Logistic Regression**:
 - Linear model that outputs probabilities.
 - Fast and easy to interpret (weights for each feature).
 - **Random Forest**:
 - Many decision trees trained on random subsets.
 - Captures nonlinear relationships and interactions.
 - Evaluation metrics:
 - **Accuracy** = (correct predictions / all predictions).
 - **Precision** = of all predicted "trustworthy," how many were correct?
 - **Recall** = of all truly "trustworthy," how many did we find?
 - **F1-score** = harmonic mean of precision and recall.
 - These baselines help us check whether the features contain useful information and give a reference level to compare more complex models.
-

3. Real-World Examples or Stories

1. **Spam vs non-spam email** Features: words used, sender, links → Label: spam (1) or not spam (0). The model learns from labeled examples and then predicts on new emails.
 2. **Medical test for disease** Features: age, blood pressure, test results → Label: disease present or not. Precision and recall matter: we care about both false positives and false negatives.
 3. **Credit card fraud detection** Features: transaction amount, location, time → Label: fraud or normal. We need a model that generalizes to new transactions it has never seen.
 4. **Our project: trust in voices** Features: pitch, HNR, jitter, shimmer, etc. → Label: neutral vs trustworthy intent. The model must generalize to new speakers, not just memorize old ones.
-

4. Terminology Explained

- **Supervised learning** – Training a model using labeled examples (X, y) so it can predict y for new X.
- **Classification** – Predicting discrete categories (0/1, A/B/C), not continuous numbers.
- **Binary classification** – Only two classes (e.g., neutral vs trustworthy).
- **Feature matrix (X)** – Table of input features (rows = examples, columns = features).
- **Label vector (y)** – One-dimensional array of target labels.
- **Train/test split** – Divide data into a training part and a testing part.
- **Overfitting** – When a model memorizes training data and performs badly on new data.
- **Generalization** – How well a model performs on unseen data.
- **Baseline model** – A simple model used as a reference point.
- **Logistic Regression** – A linear model that uses a logistic function to output probabilities for class 1.
- **Random Forest** – An ensemble of decision trees where each tree gets a random subset of data/features.

5. How It Works – Supervised Pipeline for Trust Detection

Imagine we have the feature table:

duration_seconds	mean_f0_hz	sd_f0_hz	hnr_db	shimmer_db	cpp_db	intent_label
1.65	190.3	22.1	11.0	0.34	15.2	1
1.78	175.2	18.3	9.6	0.43	13.1	0
...						

Step 1 – Split features and labels

```
X = data[["duration_seconds", "mean_f0_hz", "sd_f0_hz", "hnr_db", "shimmer_db",
"cpp_db"]]
y = data["intent_label"]
```

Step 2 – Train/test split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y
)
```

- `test_size=0.3` → 30% for testing, 70% for training.
- `stratify=y` keeps class balance similar in train and test.

Step 3 – Train Logistic Regression

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)
y_pred_log = log_reg.predict(X_test)
```

Step 4 – Evaluate Logistic Regression

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

acc_log = accuracy_score(y_test, y_pred_log)
prec_log = precision_score(y_test, y_pred_log)
rec_log = recall_score(y_test, y_pred_log)
f1_log = f1_score(y_test, y_pred_log)
```

These numbers tell us how good the model is on unseen data.

Step 5 – Train Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=0)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
```

Step 6 – Evaluate Random Forest

Same metrics as above; compare to see which does better.

Step 7 – Interpret as baselines

- If both models are around, say, 0.7–0.8 accuracy on a realistic dataset, we know acoustic features contain real signal.
- In the paper, Random Forest and Logistic Regression achieve **above-chance performance** (better than 50%) when predicting trustworthy intent from acoustic features.
- We will later refine evaluation with cross-validation (particularly leave-one-speaker-out), but this basic split is a good training step.

6. Practice Exercises

Exercise 1 – Identify the Type of Problem

We have acoustic features as numbers and want to predict `intent_label` (0 = neutral, 1 = trustworthy).

Is this: a) Regression b) Binary classification c) Multiclass classification d) Clustering

Exercise 2 – Train/Test Split Reasoning

Explain in 3–4 sentences why we should **not** train and test on the same data when building a model.

Exercise 3 – Metric Matching

Match each metric to its question:

1. Accuracy
2. Precision

3. Recall
4. F1-score

Questions: a) Of all examples that are actually trustworthy, how many did we correctly predict as trustworthy? b) Of all predictions we made as trustworthy, how many were correct? c) What fraction of all predictions (neutral + trustworthy) did we get right? d) How well do we balance precision and recall in a single number?

Exercise 4 – Confusion Matrix Interpretation

You have a binary confusion matrix:

	Predicted 0	Predicted 1
Actual 0	40	10
Actual 1	5	25

a) How many total predictions were made? b) How many true positives (TP)? c) How many true negatives (TN)? d) How many false positives (FP)? e) How many false negatives (FN)?

Exercise 5 – Comparing Models

Suppose we train two models on the same data and get:

- Logistic Regression: Accuracy = 0.72, F1 = 0.70
- Random Forest: Accuracy = 0.78, F1 = 0.77

Which model seems better overall, and why?

7. Solutions / Model Answers

Exercise 1 – Answer

Correct: **b) Binary classification** We have two classes: neutral (0) and trustworthy (1).

Exercise 2 – Sample Answer

If we train and test on the same data, the model might simply memorize the training examples instead of learning general patterns. This would make it look very accurate on the training data, but it might fail badly on new data it has never seen. Using a separate test set lets us estimate how well the model will generalize in real-world situations.

Exercise 3 – Answer

1 → c) Accuracy – fraction of all predictions that were correct. 2 → b) Precision – of all predicted trustworthy, how many were truly trustworthy. 3 → a) Recall – of all truly trustworthy examples, how many we caught. 4 → d) F1-score – a single number combining precision and recall.

Exercise 4 – Answer

Total predictions = 40 + 10 + 5 + 25 = **80**.

- True Positives (TP, actual 1 & predicted 1) = **25**.

- True Negatives (TN, actual 0 & predicted 0) = **40**.
- False Positives (FP, actual 0 but predicted 1) = **10**.
- False Negatives (FN, actual 1 but predicted 0) = **5**.

Exercise 5 – Sample Answer

The Random Forest model seems better overall, because it has higher accuracy (0.78 vs 0.72) and higher F1-score (0.77 vs 0.70). This means it is making fewer mistakes overall and has a better balance of precision and recall than Logistic Regression on this dataset.

8. Q&A – Common Student Questions

1. **Q:** Why use both Logistic Regression and Random Forest? **A:** They are different types of models. Logistic Regression is simple and linear; Random Forest is more flexible and nonlinear. Comparing them helps us see whether the relationship between features and trust is mostly linear or more complex.
2. **Q:** What does `max_iter=1000` do in Logistic Regression? **A:** It sets the maximum number of optimization steps the algorithm is allowed to take. Sometimes the default (like 100) is too low, so we increase it to ensure convergence.
3. **Q:** What is `stratify=y` in `train_test_split`? **A:** It keeps the class proportions (0s and 1s) similar in both the train and test sets, which is important for balanced evaluation.
4. **Q:** Do we always need 30% of data for testing? **A:** No, it's a common choice but not a rule. Sometimes we use 20%, 10%, or cross-validation instead. The key is to have enough data to train and enough unseen data to evaluate.
5. **Q:** Why might Random Forest perform better than Logistic Regression? **A:** Because it can model nonlinear interactions between features and handle complex decision boundaries, whereas Logistic Regression only learns a single linear boundary.
6. **Q:** What happens if our model gets 0.5 accuracy? **A:** In a balanced binary problem, that's about the same as random guessing. It suggests the model isn't learning useful patterns from the features.
7. **Q:** Do we need to scale features for Random Forest? **A:** Not usually. Tree-based models are fairly scale-invariant. However, scaling can matter a lot for Logistic Regression and other linear models.
8. **Q:** What is a "baseline" exactly? **A:** A baseline is a simple starting model that we use to measure progress. If a more complex model doesn't beat the baseline, it may not be worth the extra complexity.
9. **Q:** Can we look at feature importance in Random Forest? **A:** Yes. Random Forest provides feature importance values that show which features contribute most to splitting decisions. We'll explore interpretation in a later lesson.
10. **Q:** Is one train/test split enough? **A:** It's a start, but not ideal. In research (like the paper), we often use **cross-validation** (e.g., leave-one-speaker-out) to get more reliable estimates. That will come in a later lesson.

9. Quiz (10 Questions + Answers)

Q1. What type of learning problem is neutral vs trustworthy intent? a) Regression b) Binary classification c) Multiclass classification d) Clustering

Answer: b) Binary classification.

Q2. What is the main purpose of the test set? **Answer:** To evaluate how well the trained model generalizes to new, unseen data.

Q3. Which model outputs a probability for class 1 and then uses a threshold (usually 0.5) to decide the class? **Answer:** Logistic Regression.

Q4. In Random Forest, each tree is trained on: a) All data and all features b) Random subsets of data and features c) Only the test set d) Only neutral utterances

Answer: b) Random subsets of data and features.

Q5. Accuracy = 0.8 means: **Answer:** The model correctly classified 80% of all examples.

Q6. Which metric focuses on "Of all predicted positives, how many are correct?" a) Accuracy b) Precision c) Recall d) F1-score

Answer: b) Precision.

Q7. Which metric focuses on "Of all true positives, how many did we find?" **Answer:** Recall.

Q8. What does F1-score combine? **Answer:** It combines precision and recall into a single number (harmonic mean).

Q9. True or False: If a model has high training accuracy but low test accuracy, it may be overfitting.

Answer: True.

Q10. Why is logistic regression a good baseline model? **Answer:** It is simple, fast, interpretable, and gives a clear starting point for performance.

10. Mini Practice Project – Baseline Classifiers for Trust (with .zip)

Mini Project Title: *Training Logistic Regression & Random Forest on Synthetic Trust Data*

I've prepared a mini project folder with:

- `synthetic_trust_baseline_dataset.csv`
 - Columns:
 - `duration_seconds`
 - `mean_f0_hz`
 - `sd_f0_hz`
 - `hnr_db`
 - `shimmer_db`
 - `cpp_db`
 - `intent_label` (0 = neutral, 1 = trustworthy)

- `train_baseline_classifiers.py`
 - Every line is **fully commented** so a high school student can follow it.
 - Steps:
 - Load dataset via pandas
 - Split into train/test (`train_test_split`)
 - Train **Logistic Regression**
 - Train **Random Forest**
 - Compute accuracy, precision, recall, F1-score for each model
 - Print confusion matrices and classification reports
 - Provide hints for interpretation
- `README.txt`
 - Instructions on how to install dependencies and run the script.

How the student should use it:

1. Unzip the file.
2. Open a terminal in `lesson5_baseline_classifiers`.
3. Install packages (once):

```
pip install pandas scikit-learn
```

4. Run:

```
python train_baseline_classifiers.py
```

5. Compare:
 - Logistic Regression vs Random Forest accuracies.
 - Their precision, recall, and F1-scores.
 - How many false positives and false negatives each has.

Mini Project Extension Ideas:

- Change `test_size` (e.g., 0.2, 0.4) and see how results change.
- Try reducing feature set (only `mean_f0_hz` and `hnr_db`) and compare performance.
- Adjust Random Forest `n_estimators` (number of trees) and see if performance improves.

11. References

- scikit-learn official tutorials:
 - "Supervised learning"
 - "Classification"
 - "Model evaluation: quantifying the quality of predictions"

- Beginner-friendly resources:
 - Blog posts or videos on:
 - “Logistic Regression explained simply”
 - “Random Forest explained visually”
 - “Precision, recall, and F1-score for beginners.”
-

12. Additional Information / Teacher Tips

- **Connect back to the paper** Explain that the paper also uses **Logistic Regression and Random Forest** as main models, so the student is now doing what real researchers do—just on a smaller, synthetic dataset.
 - **Encourage interpretation** After running the script, ask:
 - Which model did better?
 - Why might that be?
 - Which feature do you think is most important?
 - **Prepare for next lessons** In later lessons, you can:
 - Introduce **cross-validation and leave-one-speaker-out**.
 - Use real feature tables from the paper.
 - Explore **feature importance** and model interpretation.
-

Lesson 6 – Cross-Validation & Leave-One-Speaker-Out Evaluation

1. Lesson Summary

In this lesson, the student learns **how to evaluate trust-detection models in a fair, research-grade way**, especially for **voice data**.

We'll focus on:

- What **cross-validation** is and why it's better than a single train/test split.
- Why voice models must be evaluated in a **speaker-independent** way.
- How **Leave-One-Speaker-Out (LOSO) cross-validation** works:
 - Train on all speakers except one.
 - Test on the held-out speaker.
 - Repeat for every speaker, then average the results.
- How to implement LOSO using **scikit-learn's `LeaveOneGroupOut`**.

This mirrors what the research paper does when evaluating models on the trustworthy-intent dataset: they want to know if a model can predict trust **for new voices, not just the ones it has already heard**.

2. Key Points

- A **single train/test split** is useful but can be unstable or lucky/unlucky depending on the split.

- **Cross-validation (CV)** reduces this risk by using multiple splits and averaging performance.
 - For voice data, **speaker leakage** (same speaker in train & test) can give **over-optimistic results**.
 - **Speaker-independent evaluation** means:
 - Speakers in the test set are **never** in the train set.
 - **Leave-One-Speaker-Out (LOSO)**:
 - Each fold: hold out one speaker for testing.
 - Train on all other speakers.
 - Repeat for every speaker, average metrics.
 - LOSO is stricter and more realistic for tasks like “can we detect trust in the voice of a **new** person?”
 - scikit-learn’s `LeaveOneGroupOut` makes LOSO easy when you have a `speaker_id` column.
 - Metrics (accuracy, precision, recall, F1) are computed **per fold** and then averaged.
-

3. Real-World Examples or Stories

1. **Teacher grading new students** If you design a test and give practice questions, you want to know: “Will this test work for next year’s students who weren’t in my practice group?” LOSO mimics that: we test on **one new student** at a time.
 2. **Voice assistant for new users** A company building a voice assistant must ensure it works for **people it never heard before**. If they test on the same people used for training, performance will look unrealistically high.
 3. **Coach picking a team strategy** A coach tries strategies across different games, not just in one scrimmage. Cross-validation is like trying your strategy across many “games” (folds) to see if it’s robust.
 4. **Language learning app** An app that recognizes pronunciation should work for new learners, not just the ones it was trained on. LOSO is like holding out one learner at a time.
-

4. Terminology Explained

- **Cross-validation (CV)** – Evaluating a model by training & testing multiple times on different splits and averaging results.
 - **k-fold cross-validation** – Split the data into k equal parts; each part is used as test once, and we average across k runs.
 - **Groups** – A column (like `speaker_id`) that tells us which rows belong together.
 - **Speaker leakage** – When recordings from the same speaker appear in both training and test sets; the model may “recognize the speaker” instead of learning general patterns.
 - **Speaker-independent evaluation** – Evaluation where all speakers in the test set are different from those in the training set.
 - **Leave-One-Group-Out (LOGO / LOSO)** – A form of cross-validation where each group (here, each speaker) is left out once as the test set.
 - **Fold** – One round of train/test split in cross-validation.
 - **Mean ± standard deviation of metrics** – Average performance and how much it varies from fold to fold.
-

5. How It Works – LOSO in Practice

We have a dataset with columns like:

speaker_id	duration_seconds	mean_f0_hz	sd_f0_hz	hnr_db	shimmer_db	cpp_db	intent_label
S01	1.68	180.2	18.5	9.6	0.42	13.1	0
S01	1.61	192.3	22.0	11.2	0.35	15.0	1
...							
S16	...	...	...	...	...	...	
...							

We want to test: **If we train on 15 speakers, can we predict trust for the 16th (new) speaker?**

Step 1 – Prepare X, y, and groups

```
feature_columns = ["duration_seconds", "mean_f0_hz", "sd_f0_hz",
                  "hnr_db", "shimmer_db", "cpp_db"]
X = data[feature_columns]      # features
y = data["intent_label"]      # labels
groups = data["speaker_id"]   # speaker IDs
```

Step 2 – Set up LeaveOneGroupOut

```
from sklearn.model_selection import LeaveOneGroupOut

logo = LeaveOneGroupOut()
```

Step 3 – Loop over folds

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

accuracies = []
precisions = []
recalls = []
f1_scores = []

fold_index = 0
for train_idx, test_idx in logo.split(X, y, groups):
    fold_index += 1

    X_train = X.iloc[train_idx]
    y_train = y.iloc[train_idx]
    X_test = X.iloc[test_idx]
    y_test = y.iloc[test_idx]
    test_speakers = groups.iloc[test_idx].unique()

    model = LogisticRegression(max_iter=1000)
```

```

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

accuracies.append(acc)
precisions.append(prec)
recalls.append(rec)
f1_scores.append(f1)

print(f"Fold {fold_index} - Test speaker(s): {list(test_speakers)}")
print("  Accuracy:", acc)
print("  Precision:", prec)
print("  Recall:", rec)
print("  F1-score:", f1)
print()

```

Step 4 – Average metrics

```

accuracies = np.array(accuracies)
print("Mean accuracy:", accuracies.mean())
print("Std accuracy:", accuracies.std())
# Similarly for precision, recall, F1

```

Interpretation:

- Each fold answers: *"How well does the model work for this new speaker?"*
- The average across folds answers: *"On average, how well do we handle new speakers?"*
- The standard deviation shows how much performance changes from speaker to speaker.

This is **exactly the kind of speaker-independent evaluation** we want for the trustworthy-intent dataset.

6. Practice Exercises

Exercise 1 – Why Speaker-Independent?

Explain in 2–3 sentences why testing on the **same speakers** you trained on can give you an overly optimistic view of model performance in a voice trust-detection task.

Exercise 2 – Comparing Evaluation Methods

List one advantage and one disadvantage of:

a) Single random train/test split b) k-fold cross-validation c) Leave-One-Speaker-Out (LOSO)

Exercise 3 – Folds in LOSO

Suppose you have 12 speakers in your dataset.

a) How many folds will LOSO create? b) In each fold, roughly what fraction of the data is used for training vs testing?

Exercise 4 – Metric Averaging

You run LOSO and get accuracy scores per fold:

```
[0.75, 0.80, 0.70, 0.85, 0.65, 0.90]
```

a) What is the mean accuracy? b) (Conceptual) What does a larger standard deviation tell you about performance across speakers?

Exercise 5 – Implementing Groups

In LOSO, we use `groups = data["speaker_id"]`.

a) What would happen if we mistakenly used `groups = data["intent_label"]`? b) Why is that wrong for speaker-independent evaluation?

7. Solutions / Model Answers

Exercise 1 – Sample Answer

If we test on the same speakers we trained on, the model can learn **speaker-specific quirks** (like a person's usual pitch) instead of general patterns of trustworthy vs neutral speech. That makes results look better than they really are—when we apply the model to a new person, performance may drop. Speaker-independent testing avoids this by always evaluating on new speakers.

Exercise 2 – Sample Answer

a) Single split

- Advantage: Simple and fast.
- Disadvantage: Results can be unstable and depend on which rows ended up in train vs test.

b) k-fold cross-validation

- Advantage: More reliable estimates by averaging over k different splits.
- Disadvantage: More computation; if we split purely by rows, we might still mix speakers between train and test.

c) LOSO

- Advantage: Strong, realistic test of generalization to new speakers.
 - Disadvantage: Can be slower (one fold per speaker) and needs enough data per speaker.
-

Exercise 3 – Answer

a) LOSO will create **12 folds**, one per speaker. b) In each fold, we train on **11/12 of the data** and test on **1/12 of the data** (the held-out speaker's utterances).

Exercise 4 – Answer

a) Mean accuracy: $[(0.75 + 0.80 + 0.70 + 0.85 + 0.65 + 0.90) / 6 = 4.65 / 6 \approx 0.775]$ So average accuracy $\approx$ **0.78** (77.5%).

b) A larger standard deviation means performance varies a lot between speakers: the model might work very well for some speakers and poorly for others, which is important to know.

Exercise 5 – Sample Answer

a) If we set `groups = data["intent_label"]`, LOSO would hold out **all neutral utterances** in one fold and all trustworthy utterances in another fold, instead of holding out one speaker at a time.

b) That is wrong because we want to test on new **speakers**, not on new labels. Using labels as groups breaks the idea of speaker-independence and makes the evaluation meaningless for the “new speaker” scenario.

8. Q&A – Common Student Questions

1. **Q:** Why not always use LOSO instead of k-fold? **A:** LOSO is great when you have a clear grouping like speakers. For other tasks (like images of cats and dogs), there may be no natural group, so regular k-fold is more appropriate.
2. **Q:** Is LOSO always better than a single split? **A:** Yes, for grouped data like this, LOSO gives a more reliable and realistic estimate, though it's more computationally expensive.
3. **Q:** Do we need to shuffle data when using LOSO? **A:** LOSO uses groups, so shuffling rows isn't as important for the split itself. But shuffling can still help when training some models internally.
4. **Q:** How many utterances per speaker do we need? **A:** More is better. With very few utterances per speaker, each test fold might be too small to get stable metrics.
5. **Q:** Can we use Random Forest with LOSO? **A:** Yes. Any classifier (Logistic Regression, Random Forest, etc.) can be used in the LOSO loop; you just fit it inside the fold loop.
6. **Q:** Why do we average metrics across folds? **A:** Each fold gives performance for one test speaker. Averaging gives an overall picture of how the model performs across all speakers.
7. **Q:** What if one speaker is much harder to predict than others? **A:** That would show up as a low accuracy for that fold and a higher standard deviation. We might investigate what's special about that speaker.
8. **Q:** Should we tune hyperparameters inside LOSO? **A:** In proper research, yes: you'd nest another cross-validation inside the training set for hyperparameter tuning. For learning purposes, we usually start with default settings.
9. **Q:** Does LOSO guarantee no overfitting? **A:** No model can fully escape overfitting, but LOSO greatly reduces over-optimistic estimates due to speaker leakage.
10. **Q:** How does this connect to the trust paper? **A:** The paper evaluates models in a **speaker-independent** manner, similar to LOSO, to demonstrate that acoustic cues of trustworthy intent generalize to new voices, not just the ones in training.

9. Quiz (10 Questions + Answers)

Q1. What is the main goal of cross-validation? **Answer:** To get a more reliable estimate of a model's performance by training and testing on multiple splits of the data and averaging the results.

Q2. In LOSO, what is left out in each fold? a) One random utterance b) One random feature c) One speaker (group) d) One class

Answer: c) One speaker (group).

Q3. Why is speaker leakage a problem? **Answer:** Because the model can learn to recognize individual speakers instead of general patterns, giving overly optimistic test performance that won't hold for new speakers.

Q4. If you have 20 speakers, how many folds does LOSO create? **Answer:** 20 folds.

Q5. True or False: In LOSO, each utterance is used in the test set exactly once.

Answer: True.

Q6. Which scikit-learn class is used for LOSO? a) `KFold` b) `StratifiedKFold` c) `LeaveOneOut` d) `LeaveOneGroupOut`

Answer: d) `LeaveOneGroupOut`.

Q7. What does the `groups` argument represent in `logo.split(X, y, groups)`? **Answer:** It represents the grouping variable (here, `speaker_id`) that tells which rows belong to each speaker.

Q8. If average accuracy is 0.80 and standard deviation is 0.02, what does that mean? **Answer:** The model's accuracy is about 80% on average, and performance is fairly consistent across speakers (only about $\pm 2\%$ variation).

Q9. If one speaker's fold has 0.50 accuracy while others are around 0.80, what might you do? **Answer:** Investigate that speaker: maybe their voice is very different, or there was a data issue. It shows the model struggles more for that speaker.

Q10. Why is LOSO especially important for the trust-in-voice dataset? **Answer:** Because we care about detecting trust for **new voices**, not just the ones we trained on. LOSO directly tests this ability.

10. Mini Practice Project – Leave-One-Speaker-Out Cross-Validation (Downloadable .zip)

Mini Project Title: *LOSO Cross-Validation for Trust from Acoustic Features*

I've created a mini project folder that includes:

- `synthetic_trust_loso_dataset.csv`
 - Columns:
 - `speaker_id` – e.g., S01, S02, ...
 - `duration_seconds`
 - `mean_f0_hz`
 - `sd_f0_hz`
 - `hnr_db`
 - `shimmer_db`
 - `cpp_db`

- `intent_label` (0 = neutral, 1 = trustworthy)
- `loso_cross_validation.py`
 - Every line is commented to explain what it does.
 - Steps:
 - Load dataset with pandas
 - Build `X`, `y`, and `groups = speaker_id`
 - Use `LeaveOneGroupOut` to create LOSO folds
 - Train Logistic Regression on each fold
 - Compute accuracy, precision, recall, F1 for each test speaker
 - Print per-speaker metrics and overall mean $\pm$ std
- `README.txt` with clear run instructions.

How the student should use it:

1. Unzip the file.
2. Open a terminal / command prompt in `lesson6_leave_one_speaker_out`.
3. Install packages if needed:

```
pip install pandas scikit-learn numpy
```

4. Run:

```
python loso_cross_validation.py
```

5. Look at:
 - Per-fold metrics (one fold per speaker).
 - Average accuracy, precision, recall, F1.
 - Standard deviations.
6. Reflect:
 - Does the model generalize similarly well to all speakers?
 - Which speaker fold is hardest? Why might that be?

11. References

- scikit-learn documentation:
 - "Cross-validation: evaluating estimator performance" (section on `LeaveOneGroupOut`).
- Introductory articles or videos on:
 - "k-fold cross-validation"
 - "Group cross-validation in scikit-learn"

- “Speaker-independent evaluation for speech models”

12. Additional Information / Teacher Tips

- **Connect to research** Point out that LOSO or similar speaker-independent protocols are standard in speech research. The student is now doing **research-style evaluation**, not just toy splits.
 - **Encourage experiments** Have the student:
 - Swap Logistic Regression for Random Forest in the script.
 - Compare average metrics and discuss why results change.
 - **Lead into next lessons** Next steps could include:
 - Feature importance analysis (which acoustic cues matter most).
 - Model interpretation (e.g., logistic regression coefficients, SHAP later).
-

Lesson 7 – Feature Importance & Interpreting the Trust Model

1. Lesson Summary

In this lesson, the student learns **how to “look inside” a model** to see **which acoustic features matter most** for predicting trustworthy vs neutral intent.

We’ll focus on:

- Why interpretation is important in research (not just “does it work?” but “**why?**”).
- How **Random Forest feature importances** work.
- How to read and reason about:
 - Which features have high importance (e.g., pitch, HNR, CPP).
 - Which features matter less (e.g., maybe duration or shimmer).
- How to connect model importance back to ideas like:

“When people try to sound trustworthy, they tend to speak with clearer, more stable voices and slightly higher pitch.”

By the end, the student will be able to **train a Random Forest on acoustic features and interpret feature importances** to support or question hypotheses in the paper.

2. Key Points

- A model that performs well is **good**, but a model we can **understand** is **better**—especially in research.
- **Feature importance** tells us which input features have the most influence on the model’s decisions.
- In **Random Forest**, importance is usually based on:
 - How much each feature reduces impurity across all trees.
- Importance is **relative**:

- A feature with importance 0.4 is more influential than one with 0.1, but the absolute numbers are less meaningful.
 - Not all importance methods are the same:
 - Tree-based importance (fast, built-in).
 - Coefficients in linear models (logistic regression).
 - More advanced methods (e.g., SHAP – later in the course / other projects).
 - Interpretation must always be combined with **domain knowledge**:
 - Does it make sense that pitch and HNR matter for trust?
 - Are any results surprising or suspicious?
 - Even with feature importance, **correlation $\neq$ causation**:
 - A feature being important does not prove it “causes” trust.
-

3. Real-World Examples or Stories

1. **Teacher grading a project** When grading, you may have a “mental model” of what matters: clarity, depth, originality. Feature importance is like listing, “Clarity: 40%, Depth: 35%, Originality: 25%.”
 2. **Car safety rating** Suppose a model predicts: “Is this car safe?” Features might include number of airbags, braking distance, stability control, etc. Feature importance might show that braking distance and airbag count matter most.
 3. **Food delivery time** Predicting delivery time: distance, time of day, driver experience, traffic conditions. Feature importance reveals which factors mostly drive delay.
 4. **Trust in voices** The paper suggests that certain voice characteristics (pitch, voice quality) are strong cues of trust. By inspecting feature importance, we check whether the model “agrees” with psychological theories and human perception.
-

4. Terminology Explained

- **Feature importance** – A score for each input feature indicating how much it contributes to model decisions.
 - **Random Forest feature_importances_** – A built-in attribute of RandomForest models in scikit-learn that gives an importance score per feature.
 - **Impurity** – A measure of how mixed classes are in a decision tree node (e.g., Gini impurity). Lower impurity = more “pure” node.
 - **Impurity decrease** – How much a split on a feature reduces impurity; more reduction → more important.
 - **Global importance** – Average importance of features over all predictions and all trees (not per individual instance).
 - **Coefficient (in logistic regression)** – Numbers multiplying each feature; large positive or negative coefficients indicate strong influence.
 - **Domain knowledge** – Expert understanding of the problem (here, how humans change their voices when being trustworthy).
-

5. How It Works – Random Forest Feature Importances

We start with the same kind of dataset you’ve seen:

duration_seconds	mean_f0_hz	sd_f0_hz	hnr_db	shimmer_db	cpp_db	intent_label
1.65	190.3	22.1	11.0	0.34	15.2	1
1.78	175.2	18.3	9.6	0.43	13.1	0
...						

Step 1 – Train a Random Forest (like in Lesson 5)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

feature_columns = ["duration_seconds", "mean_f0_hz", "sd_f0_hz",
                  "hnr_db", "shimmer_db", "cpp_db"]
X = data[feature_columns]
y = data["intent_label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y
)

rf = RandomForestClassifier(n_estimators=200, random_state=0)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
print("Test accuracy:", accuracy_score(y_test, y_pred))
```

Step 2 – Get feature importances

```
importances = rf.feature_importances_
for name, score in zip(feature_columns, importances):
    print(name, ":", score)
```

- `importances` is an array, e.g.:

```
duration_seconds : 0.07
mean_f0_hz       : 0.28
sd_f0_hz         : 0.14
hnr_db           : 0.22
shimmer_db       : 0.09
cpp_db           : 0.20
```

Interpretation:

- `mean_f0_hz` and `hnr_db` might be most important.
- This matches the idea that **pitch** and **voice clarity** are strong trust cues.

Step 3 – Sort and visualize

```
import numpy as np
import matplotlib.pyplot as plt

sorted_indices = np.argsort(importances[::-1])
sorted_names = [feature_columns[i] for i in sorted_indices]
sorted_vals = importances[sorted_indices]

plt.figure()
plt.bar(range(len(sorted_names)), sorted_vals)
plt.xticks(range(len(sorted_names)), sorted_names, rotation=45)
plt.title("Random Forest Feature Importances")
plt.ylabel("Importance")
plt.tight_layout()
plt.show()
```

Step 4 – Connect to the paper

- If the paper reports that certain acoustic features are particularly predictive (e.g., increased F0, increased HNR, changed speaking rate), we can see whether our feature importance ranking aligns with those findings.
- This is a key step in “reproducing and understanding” the paper, not just copying code.

6. Practice Exercises

Exercise 1 – Reading Importances

Suppose the model prints:

```
duration_seconds : 0.08
mean_f0_hz       : 0.30
sd_f0_hz         : 0.10
hnr_db           : 0.25
shimmer_db       : 0.07
cpp_db           : 0.20
```

a) Which feature is most important? b) Which feature is least important?

Exercise 2 – Ranking Features

Based on the importances above, list the features from **most important to least important**.

Exercise 3 – Interpretation

Using the same importances, write 2–3 sentences explaining what the model thinks are the most important cues for trustworthy vs neutral speech.

Exercise 4 – Importance vs Performance

True or False: “If one feature has high importance, then the model’s accuracy must also be very high.”

Explain your answer briefly.

Exercise 5 – Domain Knowledge Check

Suppose the model says `duration_seconds` is by far the most important feature, much higher than pitch or HNR.

a) What potential issues might this indicate? b) What follow-up steps could you take?

7. Solutions / Model Answers

Exercise 1 – Answer

a) Most important: `mean_f0_hz` (0.30). b) Least important: `shimmer_db` (0.07) (tied with `duration_seconds`, but shimmer is slightly lower in this example).

Exercise 2 – Answer

From most to least important:

1. `mean_f0_hz` (0.30)
 2. `hnr_db` (0.25)
 3. `cpp_db` (0.20)
 4. `sd_f0_hz` (0.10)
 5. `duration_seconds` (0.08)
 6. `shimmer_db` (0.07)
-

Exercise 3 – Sample Answer

The model relies most on `mean_f0_hz` (average pitch), followed by `hnr_db` (voice clarity) and `cpp_db` (a voice quality measure). This suggests that pitch and voice quality are strong indicators of whether an utterance is trustworthy or neutral in this dataset. Duration and shimmer appear less important, meaning the model doesn't depend as heavily on how long the utterance is or small amplitude variations.

Exercise 4 – Answer

False. A feature can have high importance **relative to other features**, but the overall model accuracy might still be low if the task is hard, the dataset is noisy, or there isn't enough data. Feature importance tells us which features the model uses most, not how good the model is in absolute terms.

Exercise 5 – Sample Answer

a) If `duration_seconds` is dominating importance, it might mean:

- The dataset is biased (e.g., trustworthy utterances always longer).
- The model is "cheating" by exploiting some artifact of how data was recorded.
- Other features weren't extracted correctly.

b) Follow-up steps:

- Check distributions of duration by label; see if they're unrealistically separated.
 - Revisit feature extraction and data pre-processing.
 - Retrain with duration removed or penalized to see if results become more realistic.
-

8. Q&A – Common Student Questions

1. **Q:** Are feature importances always trustworthy? **A:** They're helpful but not perfect. They depend on the model type and how features are correlated. Use them as clues, not absolute truth.
 2. **Q:** Do feature importances tell us causation? **A:** No. They show which features the model relies on, not which features *cause* trust.
 3. **Q:** Why might a feature with small variance still be important? **A:** If small changes in that feature strongly affect the label, the model may learn to rely on it even if its variance is small.
 4. **Q:** Can we compare importances across different models? **A:** Roughly, yes, but be careful: different models calculate importance in different ways, so comparisons are not exact.
 5. **Q:** What if two features are highly correlated? **A:** Their importances can split or one can "steal" importance from the other. Interpretation gets trickier.
 6. **Q:** How do logistic regression coefficients relate to importance? **A:** Larger absolute coefficients usually mean higher importance, but they depend on feature scaling and direction (positive/negative).
 7. **Q:** Why do we still need domain knowledge if we have importances? **A:** Domain knowledge helps you judge whether the results make sense and spot weird artifacts or biases.
 8. **Q:** What is the difference between global and local explanation? **A:** Global explains importance across the whole dataset; local explains why the model made a specific prediction for one example.
 9. **Q:** Could we use SHAP values instead of Random Forest importances? **A:** Yes. SHAP is more advanced and can provide local explanations; we can introduce that later once basics are solid.
 10. **Q:** Why does the paper care about interpretation? **A:** Because we want to understand *how* humans signal trustworthy intent in their voices, not just build a black-box classifier.
-

9. Quiz (10 Questions + Answers + Explanations)

Q1. Feature importance in a Random Forest tells us: a) Exact causal effects b) How often a feature is missing c) How much a feature contributes to the model's decisions d) The training time of the model

Answer: c) **Explanation:** Importances measure contribution to splits and impurity reduction, not causality or runtime.

Q2. If `mean_f0_hz` has importance 0.35 and `hnr_db` has 0.25, what can we say? **Answer:** `mean_f0_hz` is more influential than `hnr_db` in the model's decisions, but both are relatively important.

Q3. True or False: "Feature importances always sum to 1 in scikit-learn's RandomForest."

Answer: True. **Explanation:** They are normalized to sum to 1.

Q4. Which of the following is *not* a voice-related feature? a) `mean_f0_hz` b) `hnr_db` c) `height_cm` d) `shimmer_db`

Answer: c) **Explanation:** The others are acoustic; height is not.

Q5. If all features have similar importance values, what might that mean? **Answer:** The model is using all features somewhat evenly; no single feature dominates, or features may be correlated in a way that shares importance.

Q6. In this trust detection scenario, which features are most likely to be important according to domain knowledge? a) Random ID numbers b) Pitch and voice quality measures c) File name length d) Recording index

Answer: b) **Explanation:** Research suggests pitch and voice quality carry trust cues.

Q7. If a feature importance is 0, what does that usually mean? **Answer:** The model did not use that feature in any split and found it irrelevant for predictions.

Q8. What is a good next step after seeing feature importances? a) Immediately delete all low-importance features b) Check if the ranking matches domain expectations and inspect suspicious results c) Ignore them d) Only keep the most important feature

Answer: b) **Explanation:** We use importances for insight, not automatic pruning without thinking.

Q9. True or False: "High feature importance guarantees high accuracy on new speakers."

Answer: False. **Explanation:** Importance describes relative influence; generalization depends on many factors and must be tested.

Q10. Why is feature importance helpful when reproducing a research paper? **Answer:** It lets us check whether our model relies on the same kinds of cues the authors describe, increasing confidence that we're replicating their findings, not just matching numbers.

10. Mini Practice Project – Feature Importance for Trust (Downloadable .zip)

Mini Project Title: *Inspecting Acoustic Feature Importance in a Trust Classifier*

I've prepared a mini project folder with:

- `synthetic_trust_feature_importance_dataset.csv`
 - Rows = utterances
 - Columns:
 - `duration_seconds`
 - `mean_f0_hz`
 - `sd_f0_hz`
 - `hnr_db`
 - `shimmer_db`
 - `cpp_db`
 - `intent_label`
- `feature_importance_trust.py`
 - Fully commented script that:
 - Loads the dataset
 - Splits into train/test
 - Trains a Random Forest
 - Computes test accuracy

- Extracts `feature_importances_`
 - Prints raw and sorted importances
 - Plots a bar chart of feature importances
- `README.txt` explaining how to run everything.

Student instructions:

1. Unzip the file.
2. Open a terminal in `lesson7_feature_importance`.
3. Install required packages (once):

```
pip install pandas scikit-learn numpy matplotlib
```

4. Run:

```
python feature_importance_trust.py
```

5. Look at:

- Test accuracy.
- Printed importance values.
- Bar chart of sorted importances.

6. Answer:

- Which feature is most important?
- Does that match your intuition about trustworthy vs neutral speech?

Optional extension:

- Remove one high-importance feature (e.g., drop `mean_f0_hz`) and re-train.
- Observe how the accuracy and remaining importances change.

11. References

- scikit-learn documentation: RandomForestClassifier and `feature_importances_`.
- Intro resources on:
 - "Random Forest feature importance"
 - "Interpreting machine learning models."
- Related conceptual follow-ups:
 - "Model-agnostic interpretation methods (e.g., permutation importance, SHAP)."

12. Additional Information / Teacher Tips

- **Link to SHAP later** This lesson sets up the idea of “which features matter?” Later, you can introduce SHAP to explain **per-utterance** contributions, tying back to your SHAP/SHapley materials from other projects.

- **Connect to psychology** Encourage the student to think:

“If the model says pitch and HNR are important, do human listeners also report these cues when judging trust?”

- **Encourage curiosity & skepticism** Ask students to find at least one result that **surprises** them and write a short paragraph exploring why it might be happening.

Lesson 8 – Human Trust Ratings & Creating Labels from Crowd Data

1. Lesson Summary

In this lesson, the student learns how to go from **raw human trust ratings** (many people rating many audio clips) to **usable labels** for machine learning.

We’ll focus on:

- How rating studies in the paper work:
 - Many raters listen to each voice clip and give a **trust rating** (e.g., 1–7 scale).
- How to organize these data in “long” format: one row = one rater’s judgment of one utterance.
- How to compute **utterance-level statistics**:
 - Mean rating
 - Standard deviation (how much raters disagree)
 - Number of ratings per clip
- How to convert continuous ratings into **binary labels** (e.g., “trustworthy” vs “neutral/low trust”) using a threshold.
- How to visualize rating distributions and disagreements.

This mirrors what the paper does conceptually: they gather human trust ratings and then create labels and summary statistics for each clip.

2. Key Points

- The dataset includes **human judgments**, not just acoustic features.
- Each utterance is rated by **multiple raters**, so we have many rows per utterance.
- We usually store ratings in **long format**:

utterance_id	rater_id	trust_rating
U001	R01	6
U001	R02	5
U001	R03	7

utterance_id	rater_id	trust_rating
...	...	...

- To use these ratings in models, we typically compute **utterance-level aggregates**:
 - Mean rating (how trustworthy this clip sounds on average).
 - Standard deviation (how much raters disagree).
 - Count of ratings.
- To train a **binary classifier**, we often convert mean ratings into 0/1 labels:
 - Example: mean ≥ 4.0 (on 1–7 scale) $\rightarrow$ label = 1 (trustworthy).
 - mean $< 4.0 \rightarrow$ label = 0 (neutral/low trust).
- We can visualize:
 - Histogram of mean ratings $\rightarrow$ how many clips are trusted or not.
 - Scatter of mean vs standard deviation $\rightarrow$ which clips are controversial (high disagreement).
- This step is crucial to reproducing the paper: you must **understand how labels are constructed** from human ratings.

3. Real-World Examples or Stories

1. **Movie ratings** Many people rate a movie 1–5 stars. We take the **average rating** to say “this movie is 4.2 stars”. In ML, we could then define “good movie” = average ≥ 3.5 .
2. **Teacher evaluations** Students rate a teacher from 1–7. The school might use the **mean rating** to summarize teaching quality and look at **disagreement** (some students strongly like, others dislike).
3. **Restaurant review scores** Thousands of people give 1–5 star ratings. Sites display the mean rating and sometimes highlight restaurants with both high mean and many ratings.
4. **Trust in voices** In the paper, listeners rate “how trustworthy does this voice sound?” We need to combine many ratings per utterance into a single score or label to train ML models.

4. Terminology Explained

- **Long format** – Each row is one rater’s rating for one item (here, one voice clip).
- **Aggregated ratings** – Combining many ratings for the same item into summary numbers (mean, std, count).
- **Mean rating** – Average of all ratings for one utterance.
- **Standard deviation (std)** – A measure of how spread out ratings are; higher std = more disagreement.
- **Thresholding** – Choosing a cutoff (e.g., mean ≥ 4.0) to turn continuous ratings into categories.
- **Binary label** – A label with only two values (e.g., 0 = neutral/low trust, 1 = trustworthy).
- **Inter-rater agreement** – How much raters agree with each other (we get a sense of this from std).

5. How It Works – From Ratings to Labels

We start with a ratings table like this (long format):

--

utterance_id	rater_id	trust_rating
U001	R01	6
U001	R02	5
U001	R03	7
U002	R01	3
U002	R02	4
...		

Step 1 – Load and inspect

```
import pandas as pd

ratings = pd.read_csv("synthetic_trust_ratings_long.csv")
print(ratings.head())
print(ratings.shape) # (rows, columns)
print("Unique utterances:", ratings["utterance_id"].nunique())
print("Unique raters:", ratings["rater_id"].nunique())
```

Step 2 – Group by utterance

We want statistics per utterance, so we group:

```
grouped = ratings.groupby("utterance_id")

aggregated = grouped["trust_rating"].agg(
    mean_rating="mean",
    std_rating="std",
    n_ratings="count"
).reset_index()

print(aggregated.head())
```

Now each row is **one utterance**:

utterance_id	mean_rating	std_rating	n_ratings
U001	6.0	1.0	12
U002	3.5	1.3	12
...			

Step 3 – Create a binary label

Define a threshold, e.g.:

```
threshold = 4.0
aggregated["intent_label"] = (aggregated["mean_rating"] >= threshold).astype(int)
print(aggregated["intent_label"].value_counts())
```

Now we have:

- `intent_label = 1` for clips rated as trustworthy on average.
- `intent_label = 0` for clips rated less trustworthy.

We can now join this `aggregated` table with our **acoustic features** table (by `utterance_id`) to train models.

Step 4 – Visualize distributions

Histogram of mean ratings:

```
import matplotlib.pyplot as plt

plt.figure()
plt.hist(aggregated["mean_rating"], bins=10)
plt.title("Histogram of mean trust ratings per utterance")
plt.xlabel("Mean rating (1-7)")
plt.ylabel("Number of utterances")
plt.tight_layout()
plt.show()
```

Scatter of mean vs disagreement:

```
plt.figure()
plt.scatter(aggregated["mean_rating"], aggregated["std_rating"])
plt.title("Mean rating vs rating disagreement (std)")
plt.xlabel("Mean rating")
plt.ylabel("Standard deviation of ratings")
plt.tight_layout()
plt.show()
```

- High mean, low std → almost everyone agrees the voice sounds trustworthy.
- Low mean, low std → everyone agrees it doesn't sound trustworthy.
- Mid mean, high std → raters disagree.

Step 5 – Save utterance-level table for modeling

```
aggregated.to_csv("trust_utterance_level_labels.csv", index=False)
```

This file will be used in later lessons to create **full model pipelines** that combine human labels and acoustic features.

6. Practice Exercises

Exercise 1 – Interpreting Mean Rating

An utterance has ratings: [5, 6, 6, 7, 5, 6]. a) What is the mean rating? b) Would you label it as “trustworthy” if the threshold is 4.0?

Exercise 2 – Interpreting Disagreement

Two utterances have these ratings:

- Utterance A: [4, 4, 4, 4, 4]
- Utterance B: [1, 7, 1, 7, 1, 7]

a) Which utterance has higher mean rating? b) Which has higher standard deviation? c) Which one shows more rater disagreement?

Exercise 3 – Counting Labels

Suppose we have this aggregated table:

utterance_id	mean_rating
U001	5.1
U002	3.8
U003	4.2
U004	2.9
U005	4.0

With threshold 4.0 ($\geq$ is label 1), what is `intent_label` for each utterance?

Exercise 4 – Threshold Experiment

For the table above, try thresholds 3.5 and 4.5. How many utterances would be labeled as 1 (trustworthy) in each case?

Exercise 5 – Why Aggregation?

Explain in 2–3 sentences why we aggregate multiple raters into one mean rating per utterance, instead of using raw rater-level rows directly in training.

7. Solutions / Model Answers

Exercise 1 – Answer

Ratings: [5, 6, 6, 7, 5, 6]

- Mean = $(5 + 6 + 6 + 7 + 5 + 6) / 6 = 35 / 6 \approx \mathbf{5.83}$.
- With threshold 4.0, mean $5.83 \geq 4.0 \rightarrow$ label as **trustworthy (1)**.

Exercise 2 – Answer

Utterance A: [4, 4, 4, 4, 4]

- Mean = 4.0, std = 0 (no spread).

Utterance B: [1, 7, 1, 7, 1, 7]

- Mean = $(1+7+1+7+1+7)/6 = 24/6 = 4.0$, but std is high (ratings alternate between extremes).

a) Both have the **same mean** (4.0). b) **Utterance B** has higher standard deviation. c) **Utterance B** shows more rater disagreement.

Exercise 3 – Answer

Threshold 4.0, label = 1 if mean_rating $\geq$ 4.0, else 0.

- U001: 5.1 $\rightarrow$ 1
 - U002: 3.8 $\rightarrow$ 0
 - U003: 4.2 $\rightarrow$ 1
 - U004: 2.9 $\rightarrow$ 0
 - U005: 4.0 $\rightarrow$ 1
-

Exercise 4 – Answer

Threshold 3.5 ($\geq$ 3.5 $\rightarrow$ 1):

- U001: 5.1 $\rightarrow$ 1
- U002: 3.8 $\rightarrow$ 1
- U003: 4.2 $\rightarrow$ 1
- U004: 2.9 $\rightarrow$ 0
- U005: 4.0 $\rightarrow$ 1

$\rightarrow$ 4 utterances labeled 1.

Threshold 4.5 ($\geq$ 4.5 $\rightarrow$ 1):

- U001: 5.1 $\rightarrow$ 1
- U002: 3.8 $\rightarrow$ 0
- U003: 4.2 $\rightarrow$ 0
- U004: 2.9 $\rightarrow$ 0
- U005: 4.0 $\rightarrow$ 0

$\rightarrow$ 1 utterance labeled 1.

Exercise 5 – Sample Answer

We aggregate multiple raters to get a **more stable and reliable** estimate of how trustworthy an utterance sounds. Individual raters may be noisy or have strong personal biases, but averaging over many of them smooths out random variation. Using one mean rating or label per utterance also makes it simpler to join with acoustic features and train a standard classifier.

8. Q&A – Common Student Questions

1. **Q:** Why not use just one rater per clip? **A:** With one rater, results are very noisy and depend strongly on that person's biases. Multiple raters give a more reliable estimate.
2. **Q:** Why use a 1–7 scale instead of 0/1 directly? **A:** A 1–7 scale captures more nuance (slightly trustworthy vs very trustworthy). We can always convert it to 0/1 later.
3. **Q:** How do we choose the threshold (like 4.0)? **A:** Often we choose the neutral midpoint of the scale, or we experiment with thresholds that balance the number of 0 and 1 labels.

4. **Q:** Why compute standard deviation of ratings? **A:** It tells us how much raters disagree. High disagreement means the clip is “controversial” or ambiguous.
 5. **Q:** Could we keep the ratings as continuous targets instead of turning them into 0/1? **A:** Yes. Then we’d do **regression** instead of classification. The paper may analyze ratings both ways.
 6. **Q:** What if some utterances have fewer ratings than others? **A:** We’d check `n_ratings` and might treat utterances with very few ratings more carefully, or exclude them.
 7. **Q:** Can we weight raters differently? **A:** In more advanced setups, yes (e.g., raters with high reliability might get more weight), but we usually start with simple averages.
 8. **Q:** Does the paper use exactly the same threshold? **A:** Not necessarily; different papers define “trustworthy” in slightly different ways. The important idea is understanding the process.
 9. **Q:** Is it okay that mean and std are real numbers while ratings are integers? **A:** Yes. Mean and std summarize the distribution; they don’t need to be integers.
 10. **Q:** How does this step help with reproducing the paper? **A:** The paper’s labels and summary statistics come from human ratings. Reproducing their analysis means replicating how they aggregated ratings and defined trust labels.
-

9. Quiz (10 Questions + Answers + Explanations)

Q1. In long format, each row corresponds to: a) One utterance b) One rater c) One [utterance, rater] rating d) One model prediction

Answer: c) **Explanation:** Long format stores one rating per row, identified by `utterance_id` and `rater_id`.

Q2. What is the main reason to compute the mean rating per utterance? **Answer:** To summarize many individual ratings into a single value that represents how trustworthy the utterance sounds on average.

Q3. Standard deviation of ratings measures: a) The average rating b) The total number of ratings c) How much raters disagree d) Whether ratings are correct

Answer: c) **Explanation:** Std quantifies spread; higher std = more disagreement.

Q4. True or False: “If all raters give exactly the same rating to an utterance, std = 0.”

Answer: True. **Explanation:** No variation → zero standard deviation.

Q5. If we set a very high threshold for trust (e.g., 5.5 on a 1–7 scale), how will that affect the number of positive labels? **Answer:** It will reduce the number of utterances labeled as trustworthy (positive), making the dataset more imbalanced.

Q6. Which of the following is *not* a reason to aggregate ratings? a) Reduce noise b) Summarize many ratings into one value c) Make it easier to combine with acoustic features d) Guarantee perfect accuracy

Answer: d) **Explanation:** Aggregation helps, but doesn’t guarantee perfection.

Q7. If an utterance has `mean_rating = 3.2` and `std_rating = 0.2`, what does that suggest? **Answer:** Raters mostly agree that the utterance is slightly below neutral in trust (low disagreement around a low-ish mean).

Q8. When we save `trust_utterance_level_labels.csv`, what is it mainly used for? **Answer:** As a label file to be joined with acoustic features so we can train and evaluate ML models.

Q9. True or False: "The choice of threshold does not affect the model at all."

Answer: False. **Explanation:** Threshold changes how labels are assigned, which changes the classification task and class balance.

Q10. In the context of this paper, why is it important to look at rating distributions, not just labels? **Answer:** Because distributions show how strong and how consistent trust judgments are. Two utterances with the same label may have very different levels of agreement and confidence.

10. Mini Practice Project – Aggregating Human Trust Ratings

Mini Project Title: *From Human Ratings to Trust Labels*

I've prepared a mini project with:

- `synthetic_trust_ratings_long.csv`
 - Columns:
 - `utterance_id` – U001, U002, ...
 - `rater_id` – R01, R02, ...
 - `trust_rating` – integer 1–7
- `aggregate_trust_ratings.py`
 - Fully commented script that:
 - Loads the ratings table
 - Computes number of utterances and raters
 - Aggregates per utterance: `mean_rating`, `std_rating`, `n_ratings`
 - Creates `intent_label` using threshold 4.0
 - Plots:
 - Histogram of mean ratings
 - Scatter of mean vs std
 - Saves `trust_utterance_level_labels.csv` for modeling.
- `README.txt` with clear instructions.

Suggested student steps:

1. Unzip the folder.
2. Install packages if needed:

```
pip install pandas numpy matplotlib
```

3. Run:

```
python aggregate_trust_ratings.py
```

4. Examine:

- The aggregated table.
- Label counts.
- Plots of mean ratings and disagreement.

5. Experiment:

- Change the threshold (e.g., 3.5, 4.5) and see how label counts change.
- Think about which threshold seems more reasonable.

11. References

- General reading on:
 - "Likert scales and rating aggregation."
 - "Inter-rater reliability and agreement."
- Example tutorials that discuss:
 - Converting human ratings to labels for ML.
 - Using `groupby` in pandas for aggregation.

12. Additional Information / Teacher Tips

- **Tie back to the paper** Emphasize that the paper's labels and analysis start from **exactly this type of ratings data**. Understanding this step gives the student a deeper appreciation of the dataset.
- **Connect to statistics gently** You can briefly mention more advanced reliability metrics (like Cronbach's alpha or ICC) as future topics, but keep this lesson focused on intuitive measures (mean and std).
- **Bridge to next lessons** Next, we can:
 - Join utterance-level labels with acoustic features.
 - Recreate something close to the paper's **full pipeline**: LOSO, logistic regression, random forest, metrics, and comparison to human ratings.

Lesson 9 – End-to-End Speaker-Independent Trust Classification Pipeline

1. Lesson Summary

This lesson pulls everything together into a **single, end-to-end pipeline** that looks a lot like what the research paper is doing:

1. We start from an utterance-level dataset that already has:
 - **Speaker IDs**
 - **Acoustic features**
 - **Aggregated human trust ratings** (mean, std, count)
 - **Binary trust label** (`intent_label`)
2. We then:
 - Build **feature matrix (X)**, **labels (y)**, and **groups (speaker_id)**.
 - Run **Leave-One-Speaker-Out (LOSO)** cross-validation.
 - Train **two models**: Logistic Regression and Random Forest.
 - Compute **accuracy, precision, recall, and F1** for each held-out speaker.
 - Average metrics across speakers and compare models.

By the end of this lesson, the student will understand how to combine **human labels + acoustic features + LOSO + models + metrics** into a coherent, research-style evaluation pipeline for trust in voices.

2. Key Points

- An **end-to-end pipeline** connects all steps from data to evaluation: ratings → labels → features → splitting → training → metrics.
 - **Speaker-independent evaluation** (LOSO) is baked into the pipeline via `LeaveOneGroupOut`.
 - We compare **at least two models** (Logistic Regression vs Random Forest) to see:
 - Which performs better.
 - How model complexity affects generalization.
 - We look at **average performance** across all speakers and **variation** (std).
 - This is the bridge between “**toy scripts**” and **reproducing a real paper’s results**.
 - Once this pipeline is clear, swapping in the **real dataset and code from the paper** is mostly an engineering task, not a conceptual one.
-

3. Real-World Examples or Stories

1. **Tournament scoring** A sports league doesn’t decide the champion from one game; it tracks scores across all matches and all teams. Our pipeline is like the league table: it summarizes performance across all speakers, not just one convenient split.
2. **Manufacturing QA** A factory pipeline checks raw materials, assembly, and final inspection. Each step must work. Similarly, our ML pipeline needs: clean labels, good features, fair evaluation, and clear metrics.
3. **College admissions** Colleges look at transcripts, test scores, essays, recommendations, etc. This is an end-to-end process. Our pipeline similarly looks at multiple components to judge how good a model really is.
4. **Voice-based security system** A company building a trust-sensitive voice system must test it on many speakers, not just a few. They’d build almost exactly this kind of LOSO pipeline internally.

4. Terminology Explained

- **End-to-end pipeline** – A full workflow from raw-ish data (features + labels) through to evaluation metrics.
- **Model family** – Type of model (e.g., Logistic Regression, Random Forest).
- **Speaker-independent** – Train on some speakers, test on different, unseen speakers.
- **Fold-level metrics** – Metrics computed on one train/test split (here, for one test speaker).
- **Aggregate metrics** – Mean and standard deviation of metrics across folds.
- **Baseline vs stronger model** – Logistic Regression is a simple baseline; Random Forest is more flexible and often stronger.

5. How It Works – Building the Full Pipeline

We use a synthetic dataset that has one row per **utterance**:

```
utterance_id | speaker_id | duration_seconds | mean_f0_hz | sd_f0_hz | hnr_db |
shimmer_db | cpp_db | mean_rating | std_rating | n_ratings | intent_label
-----
U001         | S01        | 1.63           | 188.4      | 21.7     | 10.9   | 0.38
| 14.9       | 4.83         | 0.61          | 12         | 1        |
...
```

Step 1 – Load and inspect

```
import pandas as pd

data = pd.read_csv("synthetic_trust_full_pipeline_dataset.csv")
print(data.head(8))
print("Shape:", data.shape)
print("Speakers:", data["speaker_id"].nunique())
print("Utterances:", data["utterance_id"].nunique())
print("Label counts:", data["intent_label"].value_counts())
```

Step 2 – Build X, y, groups

```
feature_columns = ["duration_seconds", "mean_f0_hz", "sd_f0_hz",
                  "hnr_db", "shimmer_db", "cpp_db"]

X = data[feature_columns]          # acoustic features
y = data["intent_label"]           # 0/1 trust label
groups = data["speaker_id"]        # speaker IDs for LOSO
```

Step 3 – Set up LOSO

```
from sklearn.model_selection import LeaveOneGroupOut

logo = LeaveOneGroupOut()
```

Step 4 – Loop over speakers; train two models per fold

```

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

log_accs, log_precs, log_recs, log_f1s = [], [], [], []
rf_accs, rf_precs, rf_recs, rf_f1s = [], [], [], []

fold_index = 0

for train_idx, test_idx in logo.split(X, y, groups):
    fold_index += 1
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
    test_speakers = groups.iloc[test_idx].unique()

    # Logistic Regression
    log_model = LogisticRegression(max_iter=1000)
    log_model.fit(X_train, y_train)
    y_pred_log = log_model.predict(X_test)

    log_acc = accuracy_score(y_test, y_pred_log)
    log_prec = precision_score(y_test, y_pred_log)
    log_rec = recall_score(y_test, y_pred_log)
    log_f1 = f1_score(y_test, y_pred_log)
    log_accs.append(log_acc); log_precs.append(log_prec)
    log_recs.append(log_rec); log_f1s.append(log_f1)

    # Random Forest
    rf_model = RandomForestClassifier(n_estimators=200, random_state=0)
    rf_model.fit(X_train, y_train)
    y_pred_rf = rf_model.predict(X_test)

    rf_acc = accuracy_score(y_test, y_pred_rf)
    rf_prec = precision_score(y_test, y_pred_rf)
    rf_rec = recall_score(y_test, y_pred_rf)
    rf_f1 = f1_score(y_test, y_pred_rf)
    rf_accs.append(rf_acc); rf_precs.append(rf_prec)
    rf_recs.append(rf_rec); rf_f1s.append(rf_f1)

    print(f"Fold {fold_index}, test speaker(s): {list(test_speakers)}")
    print("  Logistic Regression: acc", log_acc, "F1", log_f1)
    print("  Random Forest:      acc", rf_acc, "F1", rf_f1)
    print()

```

Step 5 – Aggregate metrics across all speakers

```

log_accs, log_precs, log_recs, log_f1s = map(np.array, [log_accs, log_precs, log_recs,
log_f1s])
rf_accs, rf_precs, rf_recs, rf_f1s = map(np.array, [rf_accs, rf_precs, rf_recs,
rf_f1s])

```

```

print("=== Logistic Regression (LOSO) ===")
print("Mean accuracy:", log_accs.mean(), "Std:", log_accs.std())
print("Mean F1:", log_f1s.mean(), "Std:", log_f1s.std())
print()

print("=== Random Forest (LOSO) ===")
print("Mean accuracy:", rf_accs.mean(), "Std:", rf_accs.std())
print("Mean F1:", rf_f1s.mean(), "Std:", rf_f1s.std())
print()

```

Interpretation:

- Higher mean F1 = better overall balance of precision and recall across speakers.
- Comparing LR vs RF:
 - If RF > LR, it suggests non-linear interactions between features are helpful.
 - If LR $\approx$ RF, the decision boundary may be mostly linear, or data is simple.

This is very similar to what you'd do to **reproduce the modeling section of the paper**, just with the real dataset and acoustics.

6. Practice Exercises

Exercise 1 – Why Multiple Models?

Explain in 2–3 sentences why we test both Logistic Regression and Random Forest instead of only one model.

Exercise 2 – Metrics Table

Suppose after LOSO we get:

- Logistic Regression: mean F1 = 0.68, std F1 = 0.07
- Random Forest: mean F1 = 0.74, std F1 = 0.15

a) Which model has better average F1? b) Which model's performance varies more across speakers?

Exercise 3 – Groups in LOSO

What would happen if we used `groups = utterance_id` instead of `speaker_id` in `LeaveOneGroupOut`?

Exercise 4 – Model Choice

If your goal is to write a simple, interpretable baseline for a paper, which model would you choose first and why: Logistic Regression or Random Forest?

Exercise 5 – Adding Features

Name two additional kinds of features (besides pitch and HNR) that could be added to the pipeline in a more advanced version of this project.

7. Solutions / Model Answers

Exercise 1 – Sample Answer

We test multiple models to see whether a simple linear boundary (Logistic Regression) is enough or whether a more flexible model (Random Forest) captures additional patterns. This helps us understand both **model capacity** and **dataset complexity**, instead of assuming one model is automatically best.

Exercise 2 – Answer

a) Random Forest has better average F1 (0.74 vs 0.68). b) Random Forest has higher variation (std 0.15 vs 0.07), so its performance changes more across speakers.

Exercise 3 – Answer

Using `groups = utterance_id` would make each utterance its own group, so LOSO would hold out one **utterance** at a time rather than one speaker. That would **not** enforce speaker-independence: the same speaker would still appear in both training and test sets, and we would risk optimistic performance due to speaker-specific patterns.

Exercise 4 – Answer

For a simple, interpretable baseline, I'd choose **Logistic Regression** first. It's easy to implement, fast to train, and its coefficients can be interpreted to see how each feature affects the log-odds of trust. Then I'd add Random Forest as a stronger non-linear model for comparison.

Exercise 5 – Sample Answer

Examples of additional features:

- **Prosody:** speaking rate, pause counts, energy (loudness) patterns.
 - **Spectral features:** MFCCs (Mel-frequency cepstral coefficients), spectral centroid.
 - **Linguistic features:** simple word counts or sentiment scores, if we have transcripts.
-

8. Q&A – Common Student Questions

1. **Q:** Why do we use LOSO instead of a random 80/20 split here? **A:** Because we want to test generalization to **new speakers**, not just new utterances from the same speakers. LOSO enforces that.
2. **Q:** Could we include `mean_rating` itself as a feature? **A:** For pure trust prediction from acoustics, we usually don't, because `mean_rating` is derived from human judgments. But we *can* use it to explore relationships between ratings and acoustics.
3. **Q:** Is Random Forest always better than Logistic Regression? **A:** No. Sometimes the data is simple and a linear model works just as well or better. RF is more flexible but can overfit or vary more across folds.
4. **Q:** Why do we track both mean and standard deviation of metrics across folds? **A:** Mean tells us overall performance; std tells us how stable performance is across speakers.
5. **Q:** Why do we use F1-score in addition to accuracy? **A:** Because accuracy can be misleading if classes are imbalanced. F1 balances precision and recall, especially important when wrong "trust" predictions are expensive.

6. **Q:** What if some speakers have very few utterances? **A:** Their folds may give unstable metrics. In real research, we might enforce a minimum number of utterances per speaker or analyze them separately.
7. **Q:** How close does this pipeline get to the actual paper? **A:** Conceptually it's very close: same idea of speaker-independent evaluation, similar models, similar metrics. The main differences are that the real paper uses the **actual** dataset and may include more features and models.
8. **Q:** Could we add ROC-AUC to this pipeline? **A:** Yes. We'd have the models output probability scores and then compute ROC-AUC using those, per fold and averaged.
9. **Q:** Do we need to standardize features for Logistic Regression? **A:** In many cases, yes, especially if features are on very different scales. In this simple synthetic example, we sometimes skip it for teaching, but in the real reproduction we'd likely use `StandardScaler`.
10. **Q:** Is this pipeline only for trust, or could we use it for other emotions? **A:** The same structure works for many speech tasks: friendliness, dominance, politeness, emotion categories, etc.
-

9. Quiz (10 Questions + Answers + Explanations)

Q1. The main purpose of using `speaker_id` as `groups` in LOSO is to: a) Balance class labels b) Ensure test speakers are unseen during training c) Make the code faster d) Increase the number of folds

Answer: b) **Explanation:** Groups control how data is split; using `speaker_id` ensures speaker-independent evaluation.

Q2. In this lesson's pipeline, each LOSO fold tests on: a) One random utterance b) All utterances from one speaker c) Half the dataset d) Only trustworthy utterances

Answer: b) **Explanation:** Each group = one speaker; LOSO leaves out one group per fold.

Q3. Why do we compute metrics for each fold and then average them? **Answer:** To summarize performance across all speakers and reduce the effect of any single "easy" or "hard" speaker.

Q4. True or False: "If Random Forest has a slightly higher mean F1 but much higher std, it might be less reliable across speakers than Logistic Regression."

Answer: True. **Explanation:** Higher std means more variability; some speakers may be predicted much worse.

Q5. Which model is more likely to capture complex non-linear interactions between features? a) Logistic Regression b) Random Forest

Answer: b) **Explanation:** Random Forest builds many decision trees and can represent non-linear decision boundaries.

Q6. What does `max_iter=1000` do in Logistic Regression? **Answer:** It increases the maximum number of optimization steps allowed so that the algorithm is more likely to converge.

Q7. Why is F1-score useful for trust vs neutral classification? **Answer:** It balances precision and recall, which is important when misclassifying "trustworthy" vs "not trustworthy" has asymmetrical costs.

Q8. If the label counts are very imbalanced (e.g., many more 0s than 1s), what might we consider doing? **Answer:** We might adjust class weights, collect more data, change the threshold, or use evaluation metrics that handle imbalance better (like F1 or ROC-AUC).

Q9. In this pipeline, what does `n_estimators=200` mean for Random Forest? **Answer:** The forest consists of 200 decision trees, which usually improves stability and performance.

Q10. Why is this lesson crucial for reproducing the paper? **Answer:** Because it shows how to implement the **complete modeling and evaluation process** that the paper uses, making it much easier to swap in the actual dataset and compare results.

10. Mini Practice Project – End-to-End Trust Pipeline (Downloadable .zip)

Mini Project Title: *Speaker-Independent Trust Classification Pipeline*

I've prepared a mini project folder with:

- `synthetic_trust_full_pipeline_dataset.csv`
 - Columns:
 - `utterance_id`, `speaker_id`
 - `duration_seconds`, `mean_f0_hz`, `sd_f0_hz`, `hnr_db`, `shimmer_db`, `cpp_db`
 - `mean_rating`, `std_rating`, `n_ratings`
 - `intent_label` (0 = neutral/low trust, 1 = trustworthy)
- `full_trust_pipeline_loso.py`
 - Fully commented script that:
 - Loads data and prints basic info.
 - Builds X, y, groups.
 - Runs LOSO over speakers.
 - Trains Logistic Regression and Random Forest in each fold.
 - Computes accuracy, precision, recall, F1 per fold.
 - Aggregates metrics (mean and std) for each model.
 - Prints a short comparison summary.
- `README.txt` with instructions.

Suggested student workflow:

1. Unzip the file.
2. Install dependencies (if needed):

```
pip install pandas numpy scikit-learn
```

3. Run:

```
python full_trust_pipeline_loso.py
```

4. Answer:

- Which model has higher mean F1?
- Which has higher variability across speakers?
- Does this match your expectations from earlier lessons?

5. Optional:

- Add feature standardization for Logistic Regression.
- Try changing `n_estimators` in Random Forest.
- Add ROC-AUC computation.

11. References

- scikit-learn docs:
 - `LeaveOneGroupOut`
 - `LogisticRegression`
 - `RandomForestClassifier`
 - `accuracy_score`, `precision_score`, `recall_score`, `f1_score`
- Tutorials on:
 - “Building an end-to-end ML pipeline.”
 - “Group cross-validation and speaker-independent evaluation.”

12. Additional Information / Teacher Tips

- **Mapping synthetic → real** Emphasize to the student that this synthetic project is a **dress rehearsal**. When they switch to the real paper’s dataset:
 - `synthetic_trust_full_pipeline_dataset.csv` → real features + labels file(s), joined.
 - Same LOSO logic, same metrics, same comparisons.
- **Encourage documentation** Get the student to write a short **mini-report**:
 - Data description
 - Methods (LOSO, models)
 - Results table (LR vs RF)
 - Interpretation
- **Prepare for Lesson 10** Next lesson can focus on:
 - Reproducing key numbers from the paper (as close as possible).
 - Writing up results in a **paper-style summary** (figures, tables, discussion).

Lesson 10 – Mini Replication Study & Writing Up Results

This is the “capstone” lesson: your student now acts like a **junior researcher** reproducing a simplified version of the paper’s pipeline and **writing a mini report** about it.

1. Lesson Summary

In this final lesson, the student:

- Runs a **full trust-detection experiment** on a synthetic dataset that mimics the paper's structure.
- Uses **speaker-independent LOSO evaluation** with Logistic Regression and Random Forest.
- Computes **accuracy, precision, recall, F1, and confusion matrices**.
- Generates and reads a **short automatic report**.
- Writes their own **mini replication report** describing goals, methods, results, and interpretation.

This gives them a rehearsal for reproducing and reporting the results of the real paper: they'll know not just how to code, but how to **tell the research story** clearly.

2. Key Points

- A replication is not just about running code; it's about:
 - Designing a clear **experiment**.
 - Evaluating fairly.
 - **Summarizing** results in words, tables, and maybe plots.
 - The structure of a typical report:
 - **Goal → Data → Methods → Results → Interpretation → Limitations & Future work**.
 - Confusion matrices help understand **error types**:
 - False positives (predicting trust but clip is not).
 - False negatives (missing trust).
 - The pipeline is now "modular":
 - Dataset can be swapped with the real paper's dataset.
 - Models can be changed/extended.
 - Evaluation strategy (LOSO) stays the same.
 - Good scientific practice means:
 - Being **honest** about what works and what doesn't.
 - Reporting **both mean and variability**.
 - Relating results back to **theory and prior work** (here, the paper).
-

3. Real-World Examples or Stories

1. **Science fair project report** A strong science fair project doesn't just show a cool device; it includes a **logbook**, organized **results**, and clear **conclusions**. That's exactly what you're training the student to do here.
 2. **Company internal experiments** A product team tests a new voice feature and writes an **internal experiment doc** with data, methodology, metrics, and "Ship or not?" This lesson is essentially that process for trust detection.
 3. **Academic replication** In real research, teams try to **replicate** published work on different datasets or with slightly different methods. Your student is learning the mindset and skills to do that responsibly.
-

4. Terminology Explained

- **Replication study** – An experiment that attempts to reproduce results or patterns from a previous paper, sometimes with new data or simplified settings.
 - **Confusion matrix** – A 2×2 table showing counts of:
 - True Negative (TN), False Positive (FP)
 - False Negative (FN), True Positive (TP)
 - **False positive (FP)** – Model predicts “trustworthy” when it’s actually neutral/low trust.
 - **False negative (FN)** – Model predicts “neutral/low trust” when clip is actually trustworthy.
 - **Experiment report** – A structured document explaining what you did, why, how, and what you found.
 - **Template** – A skeleton structure to help the student organize their report.
-

5. How It Works – Mini Replication Experiment

We now use a synthetic dataset `replication_trust_dataset.csv` with columns:

```
utterance_id | speaker_id | duration_seconds | mean_f0_hz | sd_f0_hz  
hnr_db | shimmer_db | cpp_db | mean_rating | std_rating | n_ratings | intent_label
```

Step 1 – Load and inspect

```
import pandas as pd  
  
data = pd.read_csv("replication_trust_dataset.csv")  
print(data.head(6))  
print("Shape:", data.shape)  
print("Speakers:", data["speaker_id"].nunique())  
print("Utterances:", data["utterance_id"].nunique())  
print("Label counts:", data["intent_label"].value_counts())
```

Step 2 – Build X, y, groups

```
feature_columns = ["duration_seconds", "mean_f0_hz", "sd_f0_hz",  
                  "hnr_db", "shimmer_db", "cpp_db"]  
  
X = data[feature_columns]  
y = data["intent_label"]  
groups = data["speaker_id"]
```

Step 3 – LOSO + two models + metrics + confusion matrices

The script:

- Uses `LeaveOneGroupOut` with `groups = speaker_id`.

- In each fold:
 - Trains Logistic Regression and Random Forest.
 - Computes accuracy, precision, recall, F1.
 - Stores predictions for a **global confusion matrix**.
- After all folds:
 - Computes mean and std of metrics.
 - Builds confusion matrices for both models.
 - Writes everything to `replication_report.txt` (a mini machine-written report).

You (or the student) then open that file and fill in a more polished version using `replication_report_template.txt`.

6. Practice Exercises

Exercise 1 – Reading the Auto Report

After running the script, open `replication_report.txt`. a) Write down the mean F1 for Logistic Regression and Random Forest. b) Which model performed better overall?

Exercise 2 – Confusion Matrix Interpretation

Suppose the Random Forest confusion matrix in the report is:

```
[[80 20]
 [18 94]]
```

Rows = true labels, columns = predicted labels. a) How many true neutral/low trust clips were misclassified as trustworthy? b) How many true trustworthy clips were misclassified as neutral/low trust?

Exercise 3 – Error Type Discussion

Using the confusion matrix above: If your system will be used to **screen audio for trustworthy-sounding voices**, which type of mistake is worse, FP or FN, and why? Answer in 2–3 sentences.

Exercise 4 – Speaker Variability

In the report, look at the std of F1 scores across folds for Random Forest. a) Is it small (e.g., < 0.05) or large (e.g., > 0.15)? b) What does that tell you about how performance differs across speakers?

Exercise 5 – Mapping to the Real Paper

List three steps from this synthetic replication that would remain **the same** when the student uses the real dataset and code from *“Human voices communicating trustworthy intent”*, and two steps that would change.

7. Solutions / Model Answers

(These are model answers; your student’s numbers will depend on their run.)

Exercise 1 – Sample Answer

a) Example (your actual numbers may differ):

- Logistic Regression mean F1 ≈ 0.70
- Random Forest mean F1 ≈ 0.77

b) Random Forest performed better overall because its mean F1 is higher.

Exercise 2 – Answer

Matrix:

```
[[80 20]
 [18 94]]
```

- Row 0, Col 1 = 20 → these are **false positives** (true 0, predicted 1).
- Row 1, Col 0 = 18 → these are **false negatives** (true 1, predicted 0).

a) 20 neutral/low trust clips misclassified as trustworthy. b) 18 trustworthy clips misclassified as neutral/low trust.

Exercise 3 – Sample Answer

If the system is used to screen for trustworthy voices, **false positives** might be more dangerous because you are incorrectly treating untrustworthy or neutral voices as trustworthy. That could lead to over-trusting someone. However, in some applications (e.g., trying not to offend people by labeling them “untrustworthy”), false negatives might also be serious. The “worse” error depends on the real-world context, which should be discussed in the project.

Exercise 4 – Sample Answer

a) If $\text{std}(F1)$ is large (e.g., > 0.15), then performance changes a lot from speaker to speaker. b) That suggests the model works well for some speakers but struggles for others. In a real replication, we might inspect which speakers are hard and why (accent, speaking style, recording quality, etc.).

Exercise 5 – Sample Answer

Same steps (synthetic vs real paper):

1. Using utterance-level labels derived from human trust ratings.
2. Using acoustic features (e.g., pitch, HNR, CPP) as model inputs.
3. Applying speaker-independent LOSO evaluation with similar metrics (accuracy, precision, recall, F1).

Different steps:

1. The real paper uses **actual raw audio + feature extraction** pipelines instead of pre-made synthetic CSVs.
 2. The real replication will use the authors’ **exact feature definitions, thresholds, and modeling details**, possibly including more advanced models or regularization settings.
-

8. Q&A – Common Student Questions

1. **Q:** Is this synthetic replication “good enough” to say we replicated the paper? **A:** It’s a **training replica**: it mirrors the structure and logic of the paper on fake data. The real replication still requires running the pipeline on the actual dataset.
 2. **Q:** Why do we write a report instead of just showing metrics in the terminal? **A:** Writing a report trains you to communicate results clearly, which is essential for competitions, papers, and internships.
 3. **Q:** Should we ever report just one metric (like accuracy)? **A:** No. It’s better to include multiple metrics (accuracy, precision, recall, F1, confusion matrix) to capture different aspects of performance.
 4. **Q:** What if Random Forest is only slightly better than Logistic Regression? **A:** That’s okay! It still tells you that a simple model is competitive, which is scientifically interesting.
 5. **Q:** Could this same pipeline be extended to multi-class labels? **A:** Yes. You’d change the labels and use multi-class metrics, but the overall structure (features, LOSO, models) remains similar.
 6. **Q:** Do I have to use LOSO forever? **A:** No, but for **speaker traits** like trust, LOSO is a strong, fair evaluation choice. Other tasks may need different strategies.
 7. **Q:** How do I handle randomness in results? **A:** Fix random seeds where possible, and report averages. Slight variation is normal; big variation is a signal to investigate.
 8. **Q:** How can I make my replication more similar to the actual paper? **A:** Use the real dataset, match the exact feature set, match model types and parameters, and compare your reported numbers with theirs.
 9. **Q:** Is it okay if my numbers don’t exactly match the paper’s? **A:** Yes. Replications often differ. The key is to **document what you did** and explore possible reasons for differences.
 10. **Q:** How does this help with ISEF or other competitions? **A:** It trains you to design proper experiments, evaluate them correctly, and present them like a real scientist—skills judges care about a lot.
-

9. Quiz (10 Questions + Answers + Explanations)

Q1. A replication study mainly aims to: a) Invent a brand-new idea b) Reproduce and test existing findings c) Avoid writing reports d) Remove human ratings

Answer: b) **Explanation:** Replication checks whether previous findings hold under similar or slightly different conditions.

Q2. A confusion matrix with many false positives means the model often: **Answer:** Predicts “trustworthy” when the true label is neutral/low trust.

Q3. True or False: “In speaker-independent evaluation, it’s okay to have the same speaker in both train and test sets.”

Answer: False. **Explanation:** That defeats the purpose; we want test speakers to be unseen.

Q4. Which is NOT typically included in an experiment report? a) Goal b) Methods c) Random song lyrics d) Results

Answer: c)

Q5. Why do we save an automatic text report (`replication_report.txt`)? **Answer:** To capture key results in a permanent, readable format that the student can analyze and use when writing their own report.

Q6. If Random Forest has higher F1 but also much higher std across folds than Logistic Regression, what might you say? **Answer:** RF is stronger on average but less stable; some speakers may be much better or worse than with LR.

Q7. In this mini replication, what do we treat as the “ground truth”? **Answer:** The binary trust labels (`intent_label`) derived from mean human trust ratings.

Q8. What is one reason to include confusion matrices in your report? **Answer:** They show how the model’s errors are distributed across classes (FP vs FN), not just overall accuracy.

Q9. True or False: “Once you have a working pipeline, replacing the synthetic dataset with the real dataset is usually straightforward.”

Answer: True. **Explanation:** Most of the code structure stays the same; you mainly change file paths and feature extraction.

Q10. Why is this lesson a good final step in the course? **Answer:** Because it requires combining everything learned—data handling, labeling, modeling, evaluation, and reporting—into a realistic mini research project.

10. Mini Practice Project – Mini Replication Study

Mini Project Title: *Mini Replication: Voice Trust Classification*

I’ve prepared a complete .zip package that includes:

- `replication_trust_dataset.csv`
- `replication_trust_experiment.py` – fully commented pipeline
- `replication_report_template.txt` – write-up template
- `README.txt` – instructions

Student instructions:

1. Unzip the folder.
2. Install requirements (if needed):

```
pip install pandas numpy scikit-learn
```

3. Run the experiment:

```
python replication_trust_experiment.py
```

4. Open `replication_report.txt` and read the summary.
5. Use `replication_report_template.txt` to write your own 1–3 page mini report:
 - Describe data, methods, results, and interpretation.
 - Reflect on how this connects to the real paper.

11. References

- scikit-learn documentation on:
 - `LeaveOneGroupOut`
 - `LogisticRegression`
 - `RandomForestClassifier`
 - `confusion_matrix` and other metrics
 - General guides on:
 - “How to write a lab report”
 - “How to write a machine learning experiment report”
-

12. Additional Information / Teacher Tips

- You can now **swap in the real dataset** from “*Human voices communicating trustworthy intent*” and adapt this pipeline:
 - Replace synthetic CSV with real utterance-level feature + label files.
 - Match the paper’s exact features and thresholding choices.
 - Compare your student’s results to reported ones (even approximate comparison is instructive).
 - For ISEF or similar competitions, this lesson sets the pattern for:
 - **Method section** (data, models, evaluation).
 - **Results section** (tables, metrics, plots).
 - **Discussion** (interpretation + limitations + future work).
-